Lecture3 数据存储架构设计



于晗

han_yu@sjtu.edu.cn

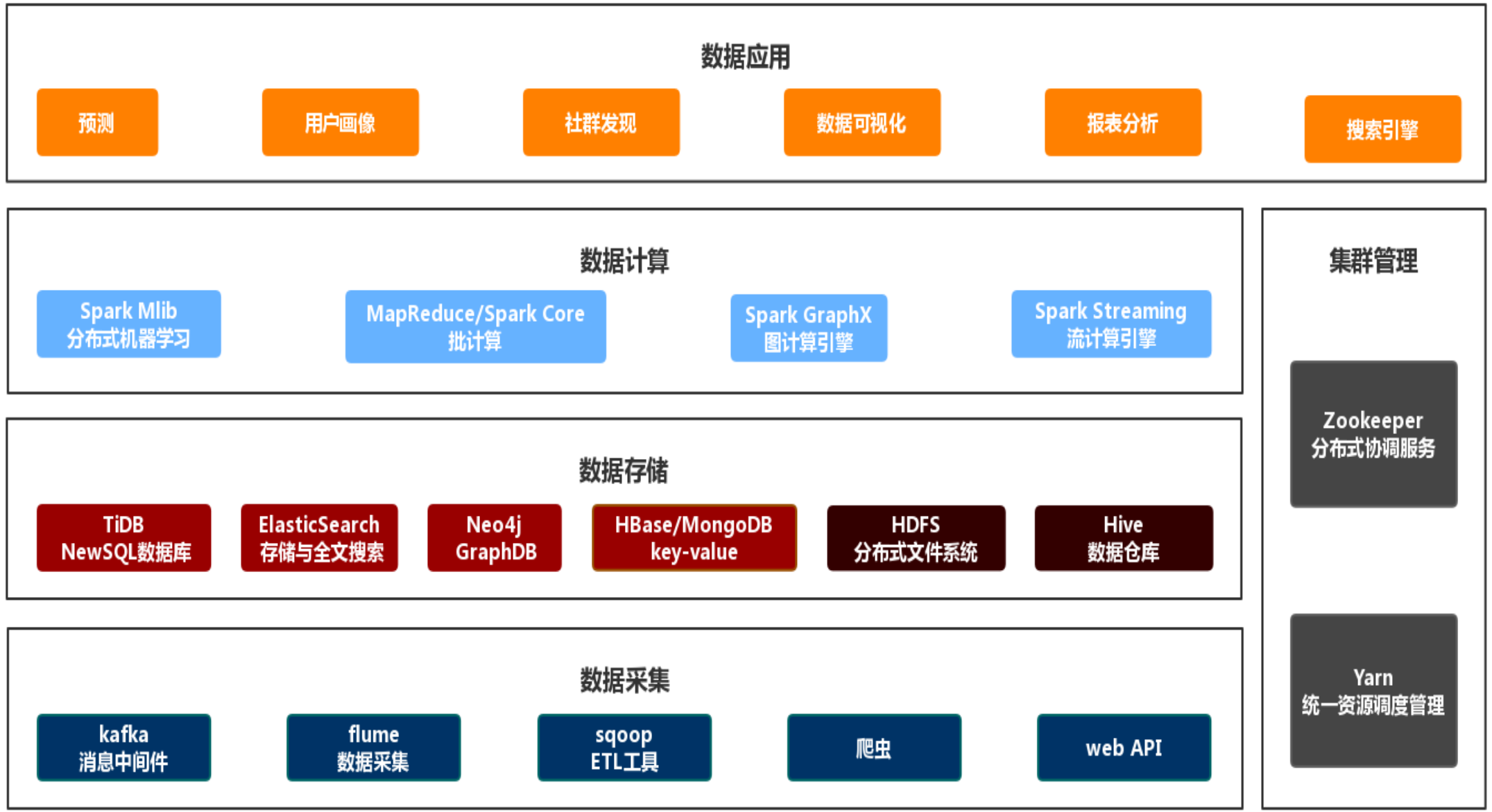
目录

- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

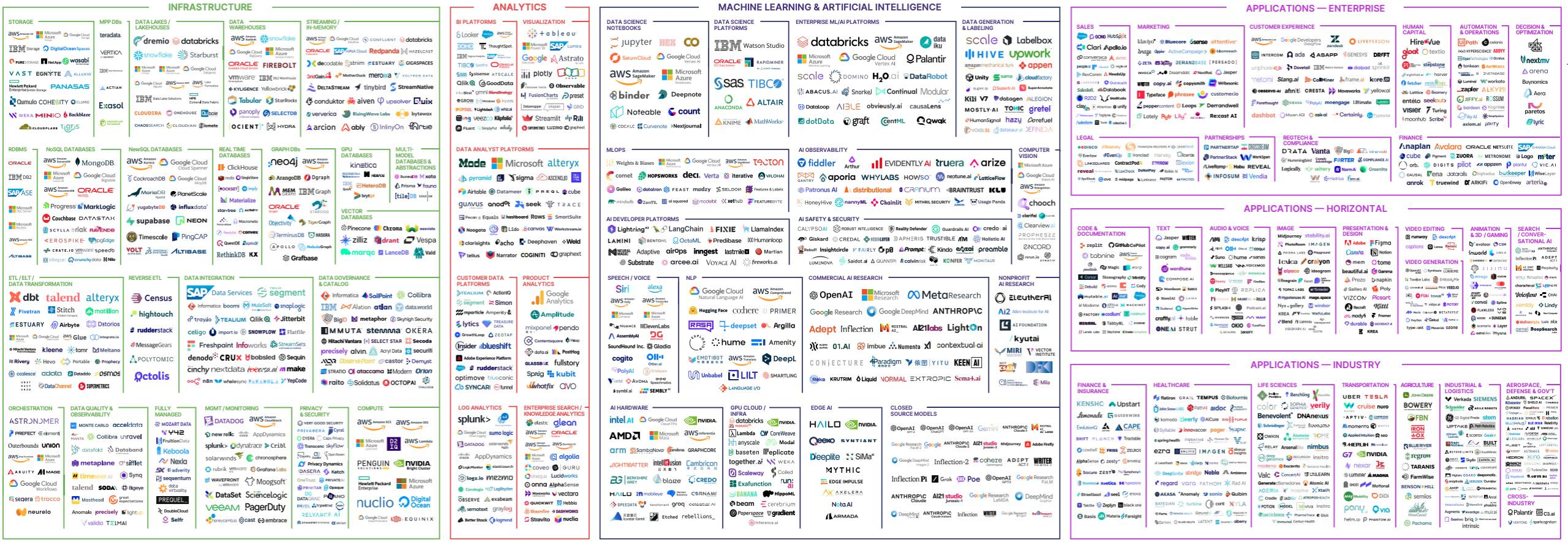
数据分析的技术框架

技术层面	功能
数据采集	利用 ETL 工具将分布的、异构数据源中的数据如关系数据、数据文件等，抽取到临时中间层后进行清洗、转换、集成，最后加载到数据仓库或数据集市中，成为联机分析处理、数据挖掘的基础； 或者也可以把实时采集的数据作为流计算系统的输入，进行实时处理分析
数据治理和存储	利用分布式文件系统、数据仓库、关系数据库、NoSQL 数据库、云数据库等，实现对结构化、半结构化和非结构化海量数据的存储和管理
数据查询与分析应用	利用分布式并行编程模型和计算框架，结合机器学习和数据挖掘算法，实现对海量数据的处理和分析；对分析结果进行可视化呈现，帮助人们更好地理解数据、分析数据
数据隐私和安全	在从大数据中挖掘潜在的巨大商业价值和学术价值的同时，构建隐私数据保护体系和数据安全体系，有效保护个人隐私和数据安全

一个典型的大数据平台架构



THE 2024 MAD (MACHINE LEARNING, ARTIFICIAL INTELLIGENCE & DATA) LANDSCAPE



Version 1.0 - March 2024
© Matt Turck (@mattturck) , Aman Kabeer (@AmanKabeer11) & FirstMark (@firstmarkcap)
Blog post: mattturck.com/MAD2024
Interactive version: MAD.firstmarkcap.com
Comments? Email MAD2024@firstmarkcap.com

FIRSTMARK
EARLY STAGE VENTURE CAPITAL

➤ https://mad.firstmark.com/

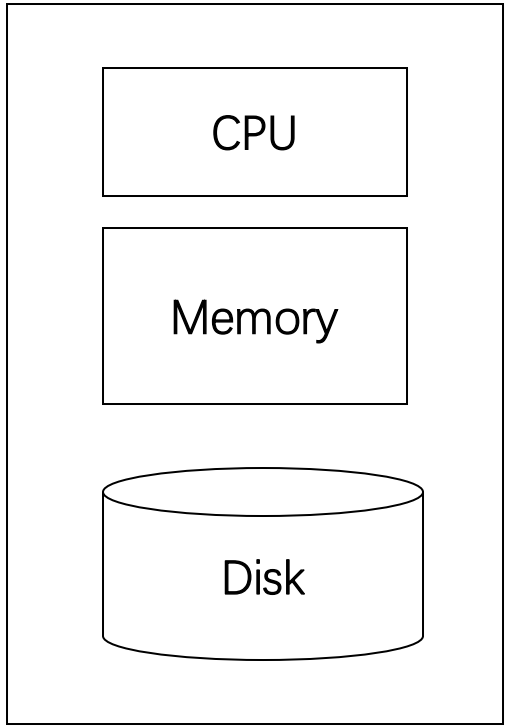
目录

- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

2.1 HDFS

机器学习，统计分析

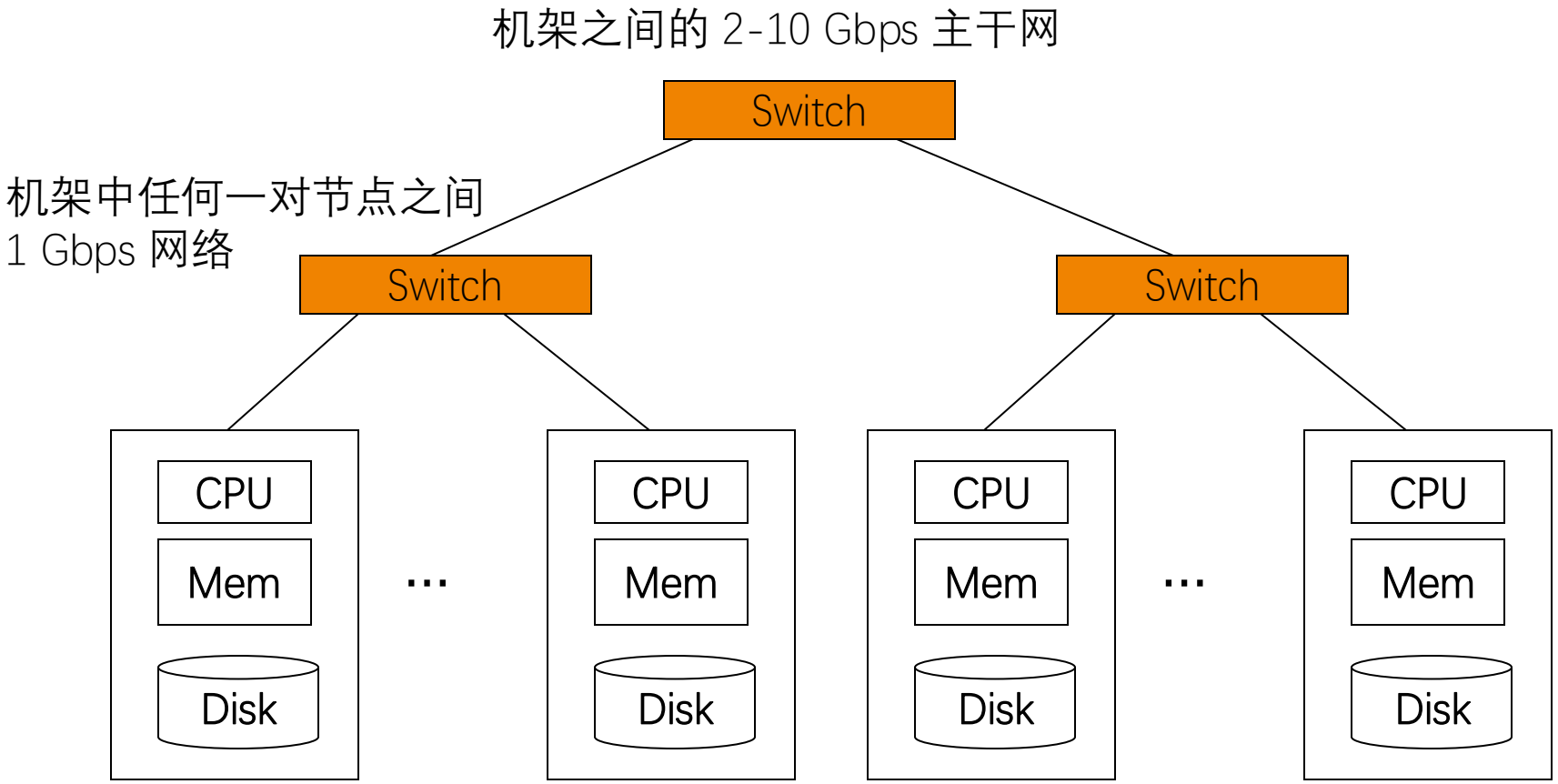
“经典”数据挖掘



单节点架构

- 大规模数据挖掘计算
- 挑战:
  - 如何分配计算?
  - 让分布式/并行编程困难
- Google例子
  - 2000万+ 网页 x 20KB = 400+ TB
  - 1 台计算机从硬盘读数据 30-35 MB/sec
    - 读取所有网页需要大约4个月
    - 存储所有网页需要大约1,000个硬盘
- 做数据分析需要花费更多时间!

集群架构



每个机架包含 16-64 个节点

2011 年，谷歌拥有 100 万台机器 (http://bit.ly/Shh0RO)



2.1 HDFS

- 在集群架构上为数据挖掘问题进行大规模计算
- 挑战：
  - 如何分配计算?
  - 如何更容易地写分布式程序?
  - 机器失效问题：
    - 一台服务器可使用 3 年(1,000天)
    - 如果有 1,000 台服务器，预计每天损失 1 台
    - 2011 年谷歌拥有约 100 万台机器
      - 每天失效1,000台机器!

解决思路

- **问题： 通过网络复制数据需要时间**
- **思路：**
  - 让计算贴近数据
  - 多次存储文件，确保可靠性
- Map-reduce 解决了这些问题
  - Google提出的计算模型/数据操作模型
  - 一种优雅的大数据工作方式
  - **存储基础设施——文件系统**
    - Google: GFS. Hadoop: HDFS
  - 编程模型
    - Map-Reduce

存储基础设施

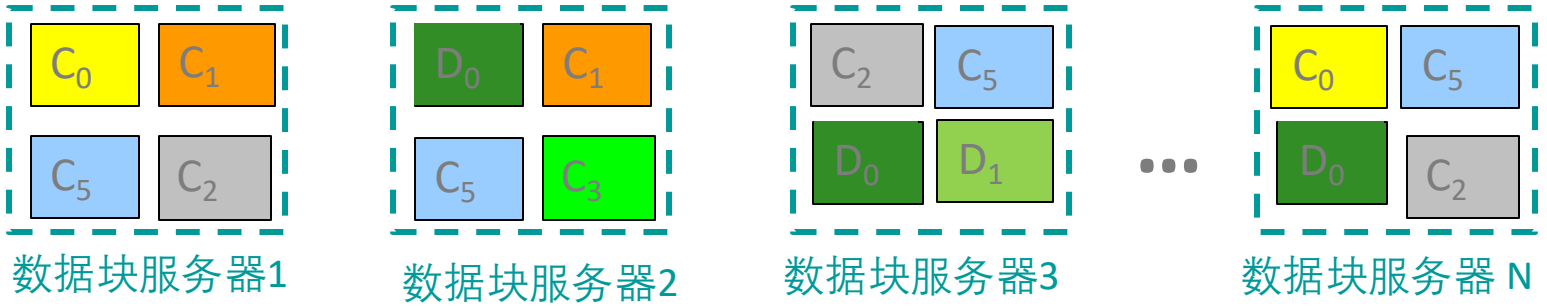
- 问题:
  - 当节点故障时，如何持久化存储数据?
- 思路:
  - 分布式文件系统:
    - 提供全局文件命名空间
    - Google GFS; Hadoop HDFS;
- 典型使用模式
  - 超大文件 (100s of GB to TB)
  - 数据几乎不就地更新
  - 读和添加操作很常见

分布式文件系统

- **数据块服务器Chunk servers**
  - 文件被划分为连续的数据块
  - 通常每个块16-64MB
  - 每个数据块复制备份(通常2-3份)
  - 尽量将副本放在不同的机架上
- **主节点**
  - 在Hadoop HDFS中被称为Name Node
  - 存储有关文件位置的元数据
  - 可能会进行备份
- **用于文件访问的客户端库**
  - 与主节点通信寻找对应数据块服务器
  - 直接与数据块服务器连接获取数据

分布式文件系统

- **可靠的分布式文件系统**
- 数据在机器间以数据块的形式传输
- 每个数据块在不同机器上备份
  - 从磁盘或机器故障中无缝恢复

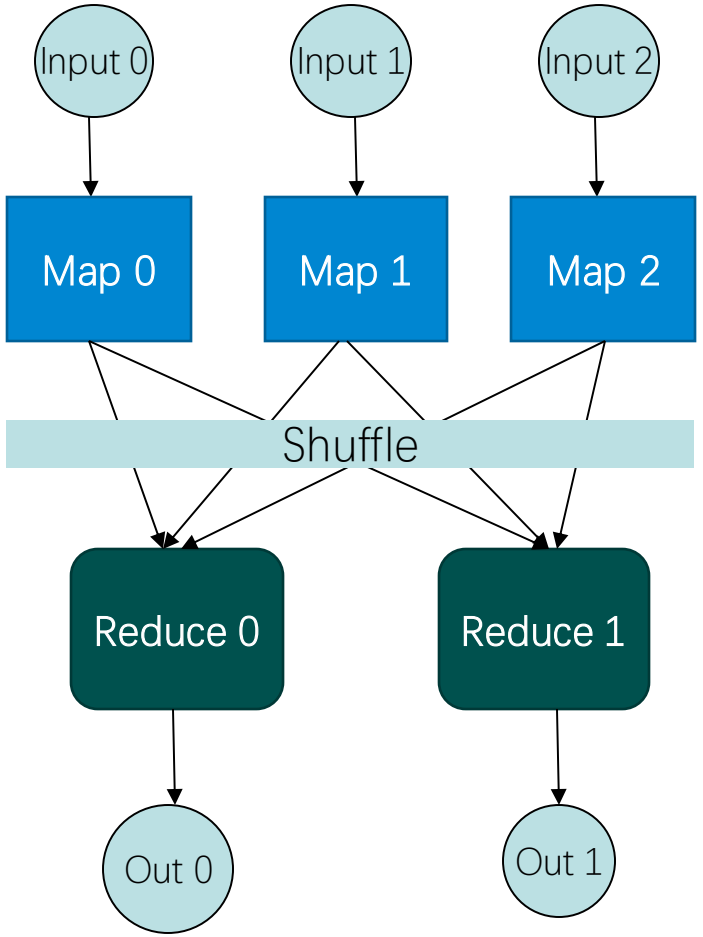


直接对数据进行计算

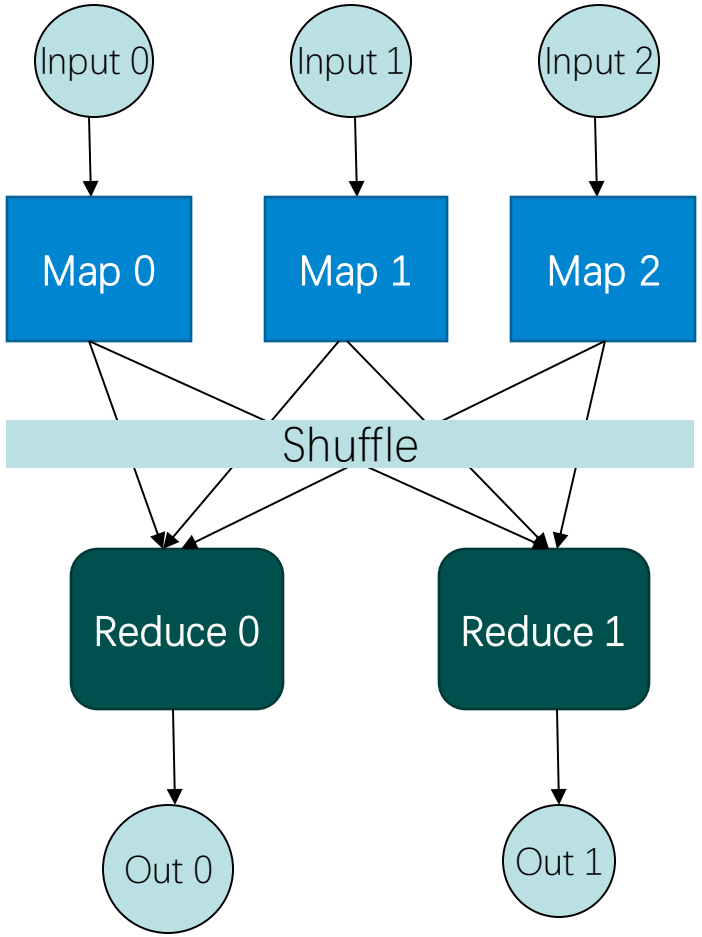
数据块服务器同时作为计算服务器

编程模型：Map-Reduce

- 编程者指定:
  - Map、Reduce函数和输入
- 工作流:
  - 读取输入，输出（key，value）集合
  - Map 将输入的kv对转化为新的k‘v‘对集合
  - 排序并打乱交给输出节点
  - 相同k’的k‘v’对被送至同一reduce函数
  - Reduce按key分组处理所有k’v’对并产生新的k’’v’’对
  - 将结果对写入文件
- 所有阶段都被划分在许多任务中

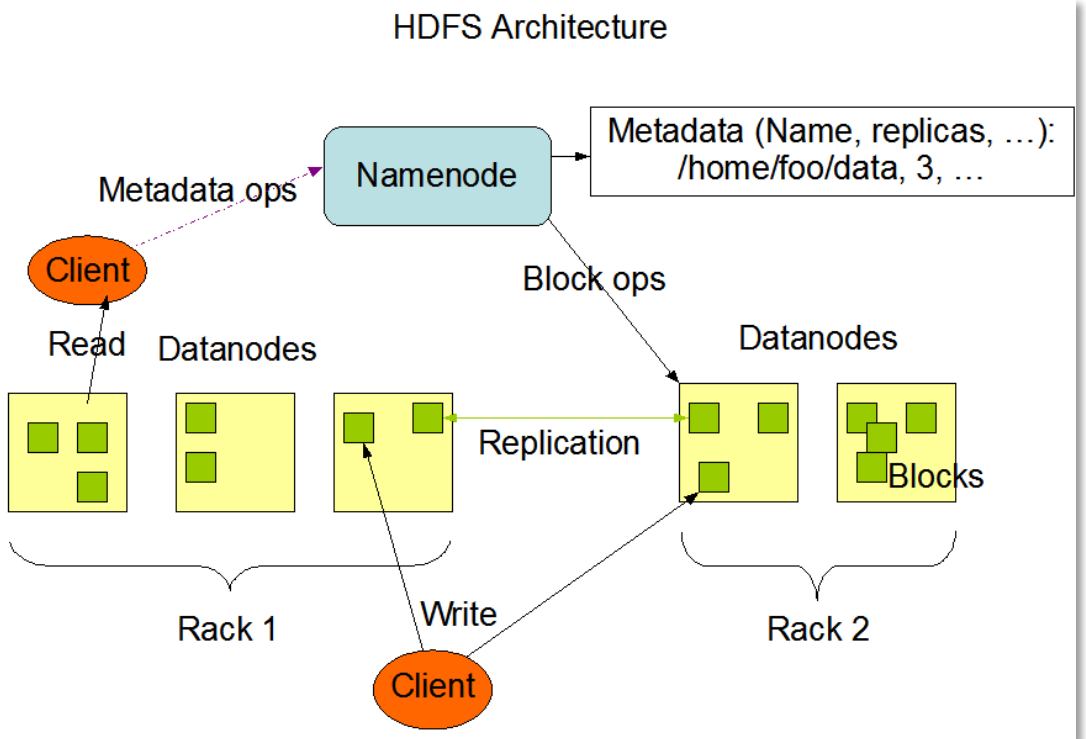


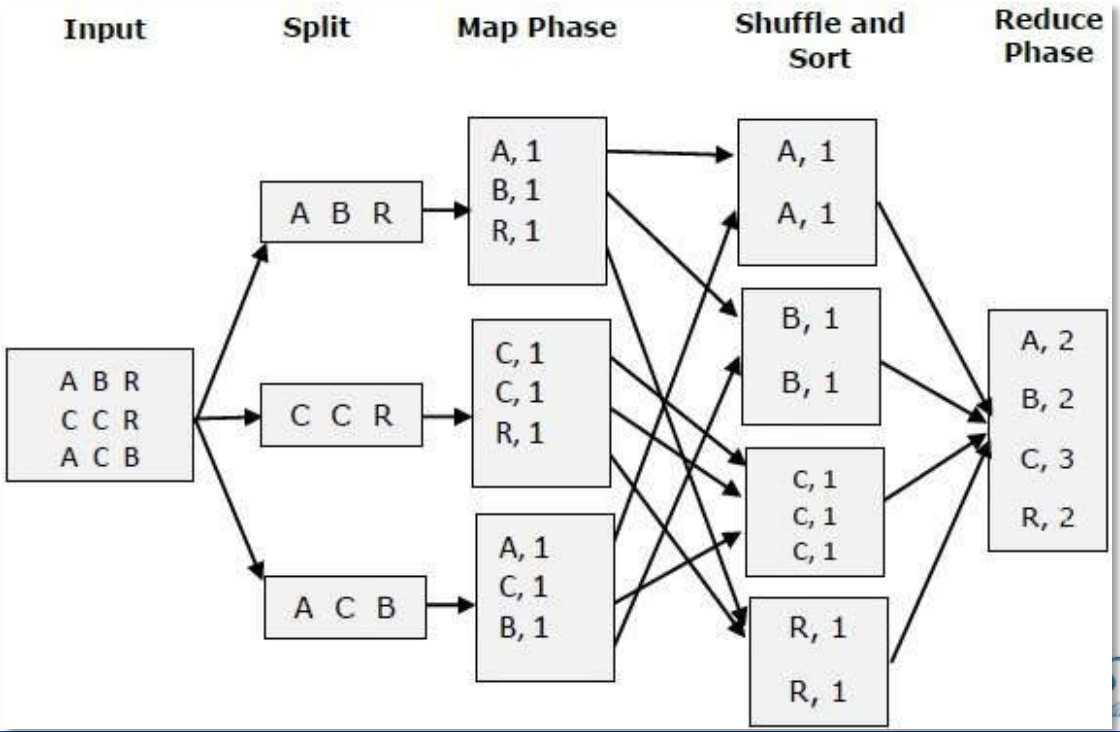
Spark


Input 0
Input 1
Input 2
Map 0
Map 1
Map 2
Shuffle
Reduce 0
Reduce 1
Out 0
Out 1

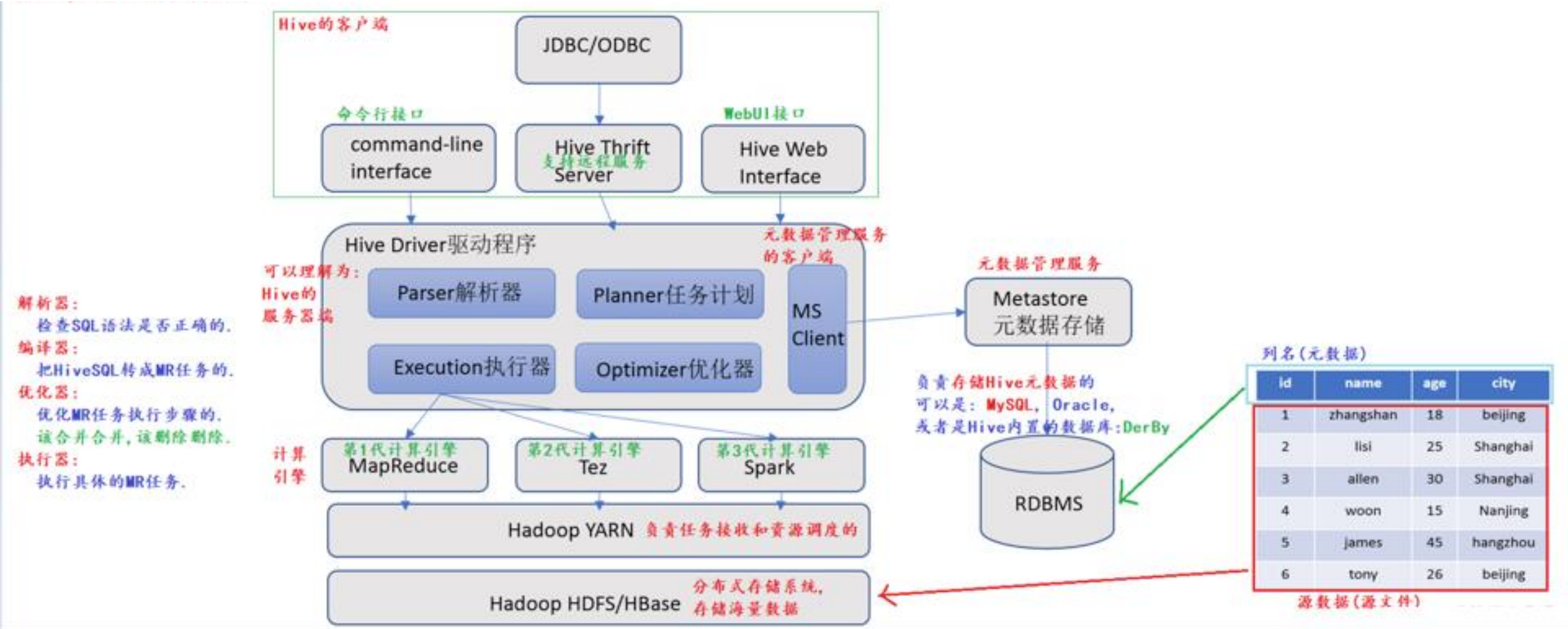
2.2 Hive：基于HDFS的数据仓库基础架构

- 直接使用hadoop面临的问题
  - 人员学习成本太高
  - 项目周期要求太短
  - MapReduce实现复杂查询逻辑开发难度太大
- Hive：利用简单的SQL语句(简称HQL)来查询、分析存储在HDFS中的数据，并把SQL语句转换成MapReduce程序来进行数据的处理。





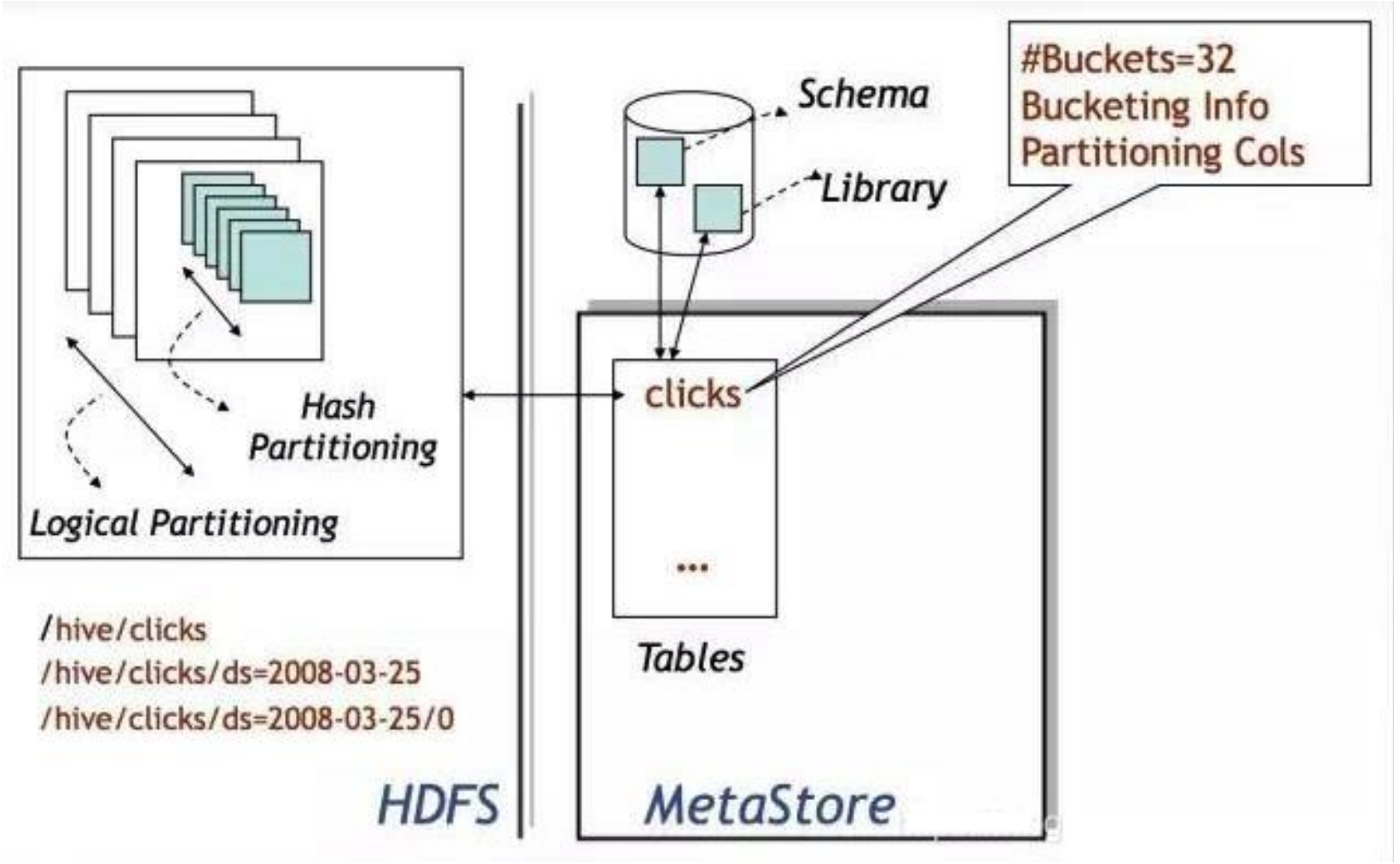
Hive架构


Hive的客户端
JDBC/ODBC
命令行接口
command-line interface
Hive Thrift Server
支持远程服务
WebUI接口
Hive Web Interface
Hive Driver驱动程序
元数据管理服务的客户端
可以理解为：Hive的服务器端
Parser解析器
Planner任务计划
Execution执行器
Optimizer优化器
MS Client
元数据管理服务
Metastore 元数据存储
负责存储Hive元数据的
可以是：MySQL，Oracle，
或者是Hive内置的数据库：DerBy
RDBMS
列名(元数据)
id
name
age
city
1
zhangshan
18
beijing
2
lisi
25
Shanghai
3
allen
30
Shanghai
4
woon
15
Nanjing
5
james
45
hangzhou
6
tony
26
beijing
源数据(源文件)
计算引擎
第1代计算引擎
MapReduce
第2代计算引擎
Tez
第3代计算引擎
Spark
Hadoop YARN
负责任务接收和资源调度的
Hadoop HDFS/HBase
分布式存储系统，存储海量数据
解析器：
检查SQL语法是否正确的.
编译器：
把HiveSQL转成MR任务的.
优化器：
优化MR任务执行步骤的.
该合并合并，该删除删除.
执行器：
执行具体的MR任务.

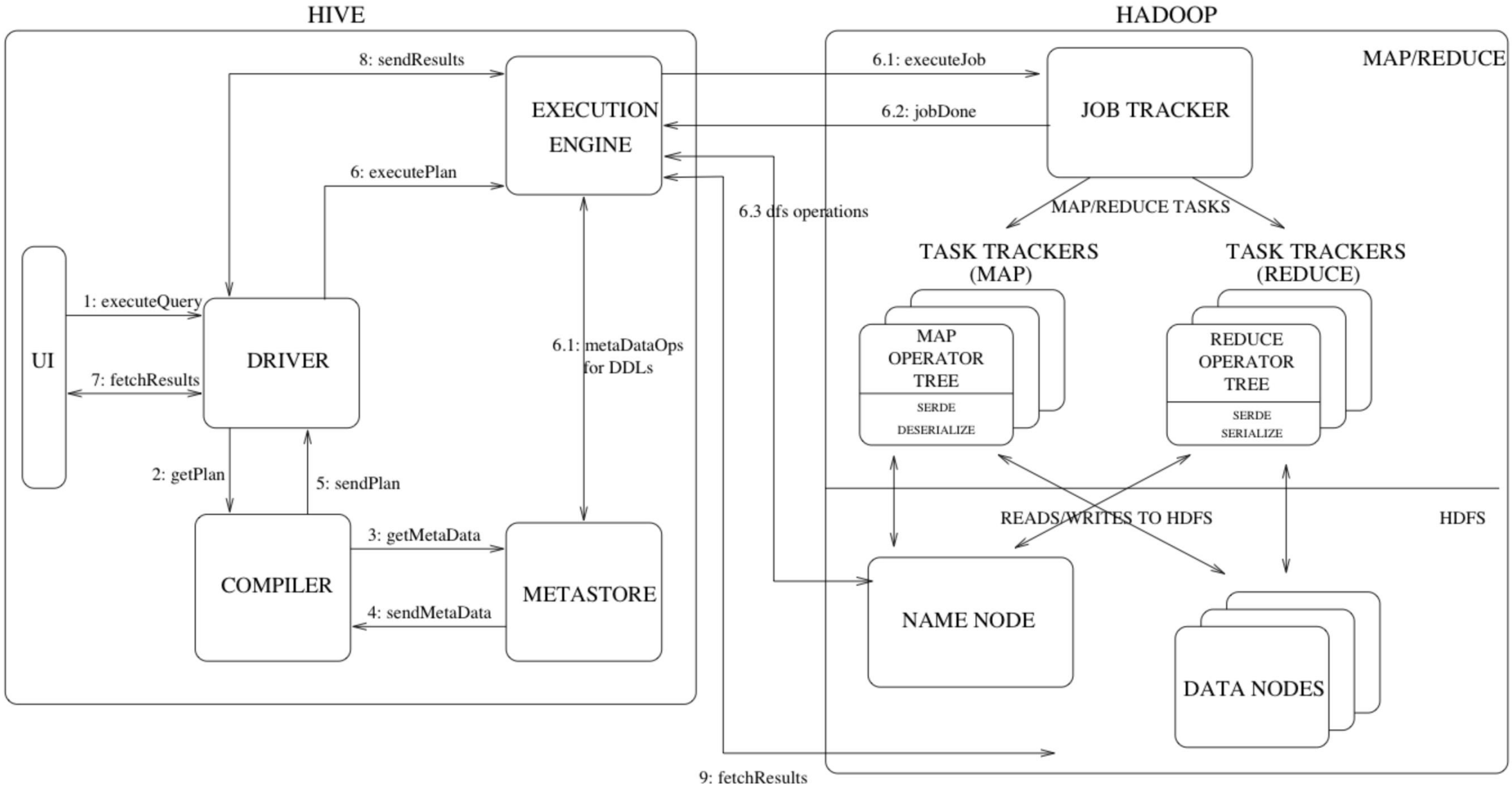
Hive数据模型

- Hive 中包含以下数据模型：Table，External Table，Partition，Bucket。

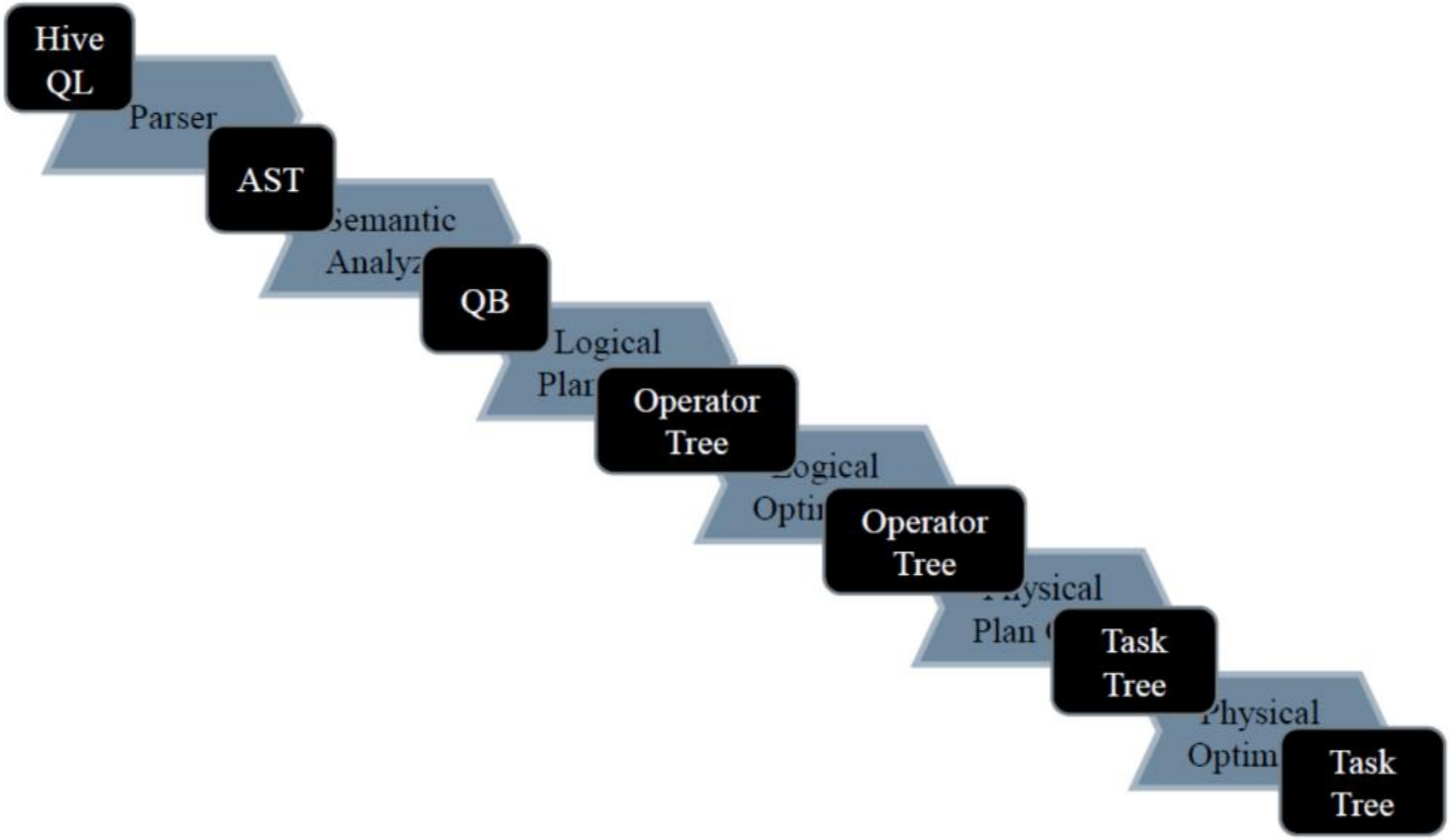
- 表table：一个表就是hdfs中的一个目录
- 区Partition：表内的一个区就是表的目录下的一个子目录
- 桶Bucket：如果有分区，那么桶就是区下的一个单位，如果表内没有区，那么桶直接就是表下的单位，桶一般是文件的形式。



Hive处理流程


HIVE
HADOOP
MAP/REDUCE
8: sendResults
6.1: executeJob
6.2: jobDone
EXECUTION ENGINE
JOB TRACKER
6: executePlan
6.3 dfs operations
MAP/REDUCE TASKS
TASK TRACKERS (MAP)
TASK TRACKERS (REDUCE)
1: executeQuery
UI
DRIVER
7: fetchResults
6.1: metaDataOps for DDLs
MAP OPERATOR TREE
SERDE DESERIALIZE
REDUCE OPERATOR TREE
SERDE SERIALIZE
2: getPlan
5: sendPlan
READS/WRITES TO HDFS
HDFS
3: getMetaData
COMPILER
METASTORE
4: sendMetaData
NAME NODE
DATA NODES
9: fetchResults

HiveQL编译过程


Hive QL
Parser
AST
Semantic Analyz
QB
Logical Plan
Operator Tree
Logical Optim
Operator Tree
Physical Plan
Task Tree
Physical Optim
Task Tree

Hive的特点

- Hive具有的特点非常适用于数据仓库
- **采用批处理方式处理海量数据**
  - Hive需要把HiveQL语句转换成MapReduce任务进行运行；
  - 数据仓库存储的是静态数据，对静态数据的分析适合采用批处理方式，不需要快速响应给出结果，而且数据本身也不会频繁变化。
- **提供适合数据仓库操作的工具**
  - Hive本身提供了一系列对数据进行提取转化加载的工具，可以存储、查询和分析存储在Hadoop中的大规模数据；
  - 非常适合数据仓库应用程序维护海量数据、对数据进行挖掘、形成意见和报告等。

Hive 与传统的数据库的对比分析

- Hive与传统数据库的区别主要体现在以下几个方面：
  - （1）数据插入：在传统数据库中同时支持导入单条数据和批量数据，而Hive中仅支持批量导入数据，因为Hive主要用来支持大规模数据集上的数据仓库应用程序的运行，常见操作是全表扫描，所以单条插入功能对Hive并不实用
  - （2）数据更新：更新是传统数据库中很重要的特性，Hive不支持数据更新。Hive是一个数据仓库工具，而数据仓库中存放的是静态数据，所以Hive不支持对数据进行更新。
  - （3）索引：索引也是传统数据库中很重要的特性，Hive在hive 0.7版本后已经可以支持索引了。但Hive不像传统的关系型数据库那样有键的概念，它只提供有限的索引功能，使用户可以在某些列上创建索引来加速一些查询操作，Hive中给一个表创建的索引数据被保存在另外的表中。

Hive 与传统的数据库的对比分析

- Hive与传统数据库的区别主要体现在以下几个方面：
  - （4）分区：传统的数据库提供分区功能来改善大型表以及具有各种访问模式的表的可伸缩性，可管理性和提高数据库效率。Hive也支持分区功能，Hive表组织成分区的形式，根据分区列的值对表进行粗略的划分，使用分区可以加快数据的查询速度。
  - （5）执行延迟：因为Hive构建于HDFS与MapReduce上，所以对比传统数据库来说Hive的延迟比较高，传统的SQL语句的延迟少于一秒，而HiveQL语句的延迟会达到分钟级。
  - （6）扩展性：传统关系数据库很难横向扩展，纵向扩展的空间也很有限。相反Hive的开发环境是基于集群的，所以具有较好的可扩展性。

Hive 与传统的数据库的对比分析

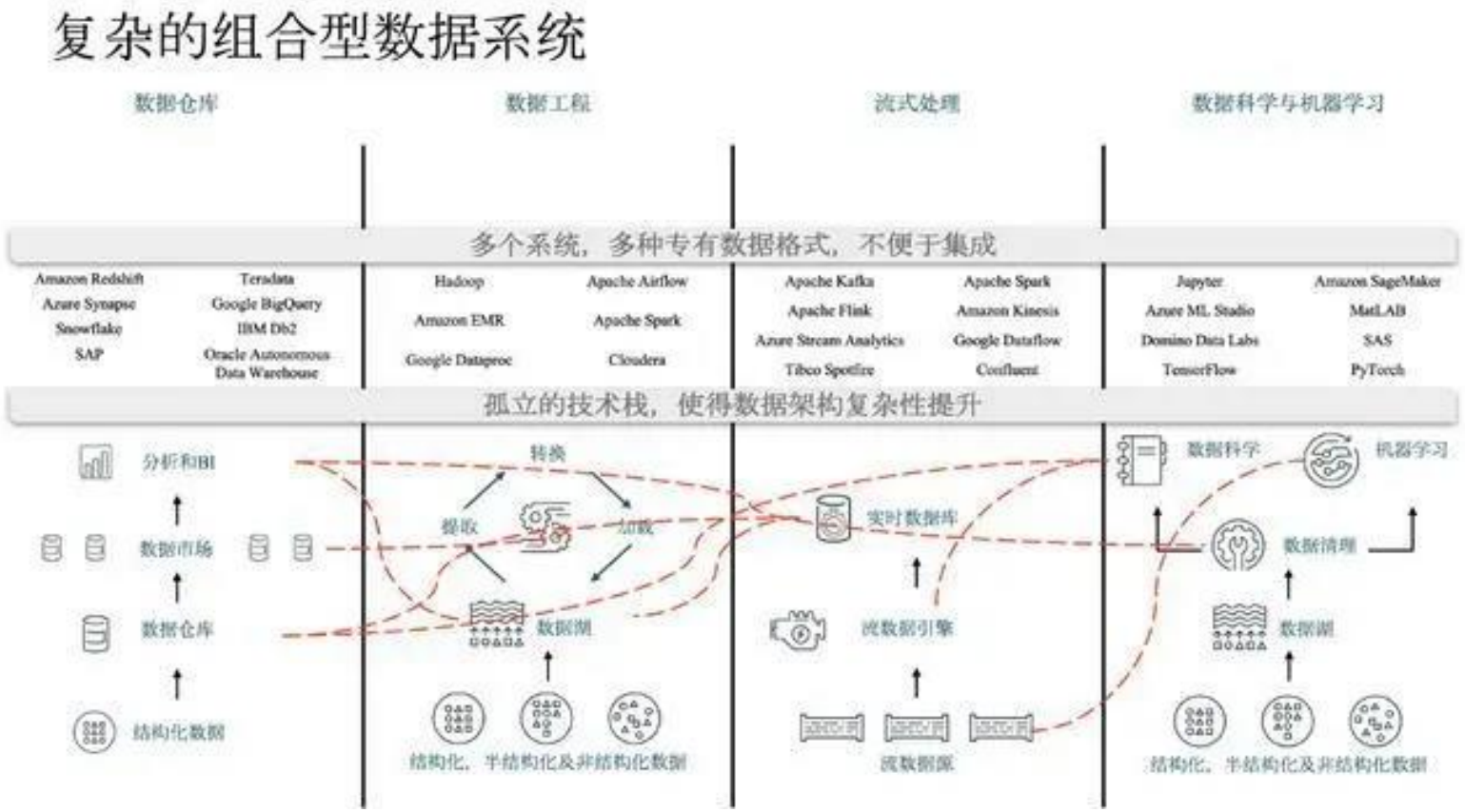
对比项	Hive	传统数据库
数据插入	支持批量导入	支持单条和批量导入
数据更新	不支持（default-0.14）	支持
索引	支持	支持
分区	支持	支持
执行延迟	高	低
扩展性	好	有限

目录

- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

3.1 Data Lake

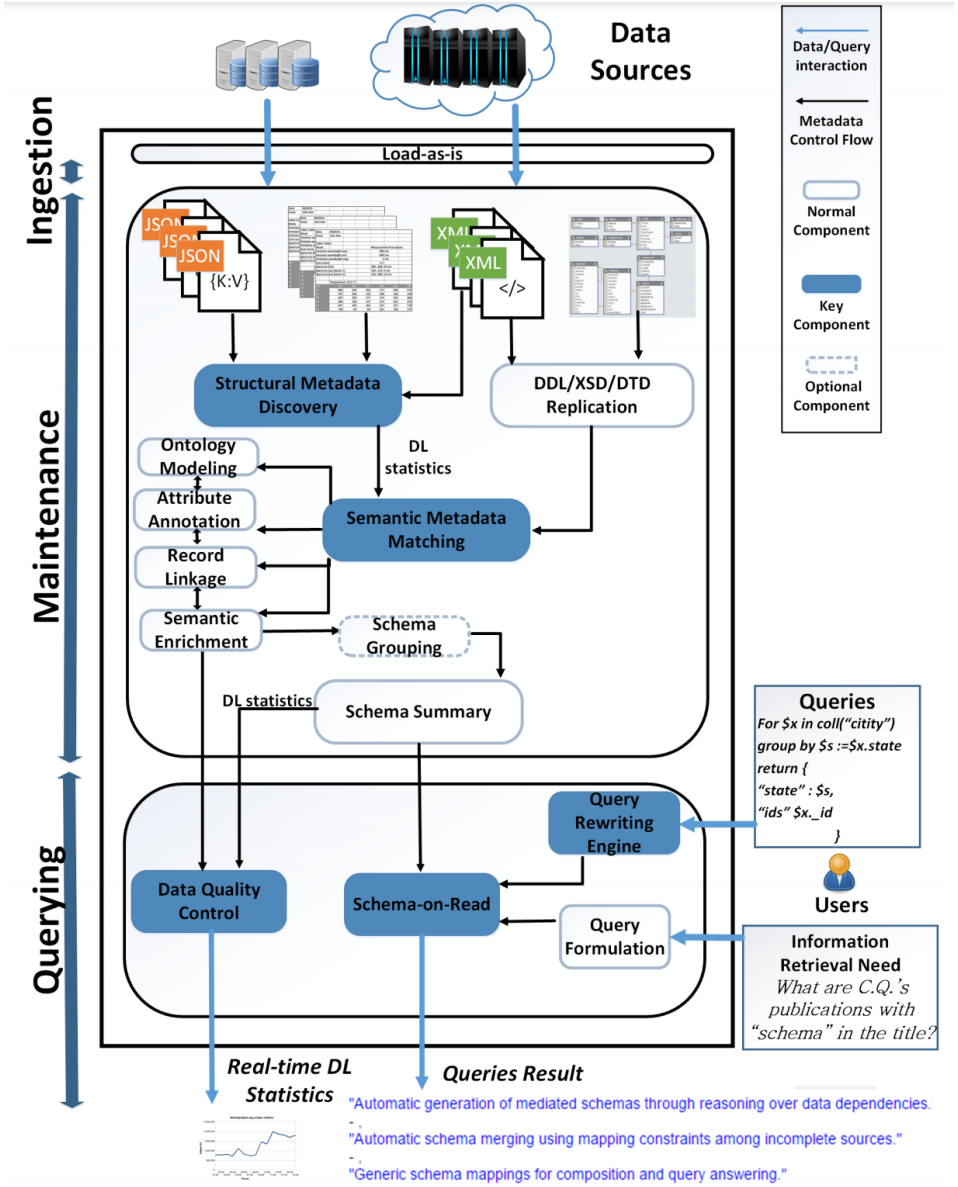
- 数据量爆炸、异构数据同时处理
- 应用需求多元化
- 智能应用需求



3.1 Data Lake

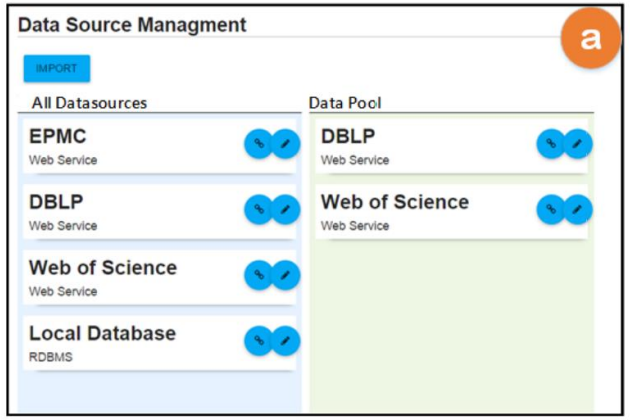
- 数据池（Data pools）
- 数据湖
- 元数据（Metadata）

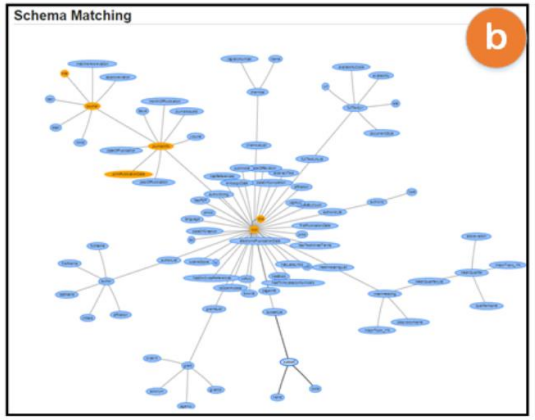
- 数据湖的特点：
  - Late binding
  - 元数据很关键

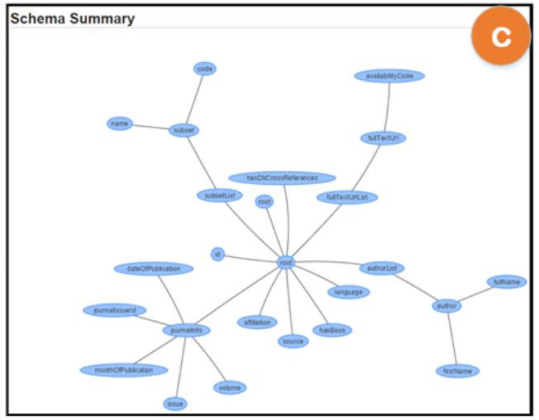


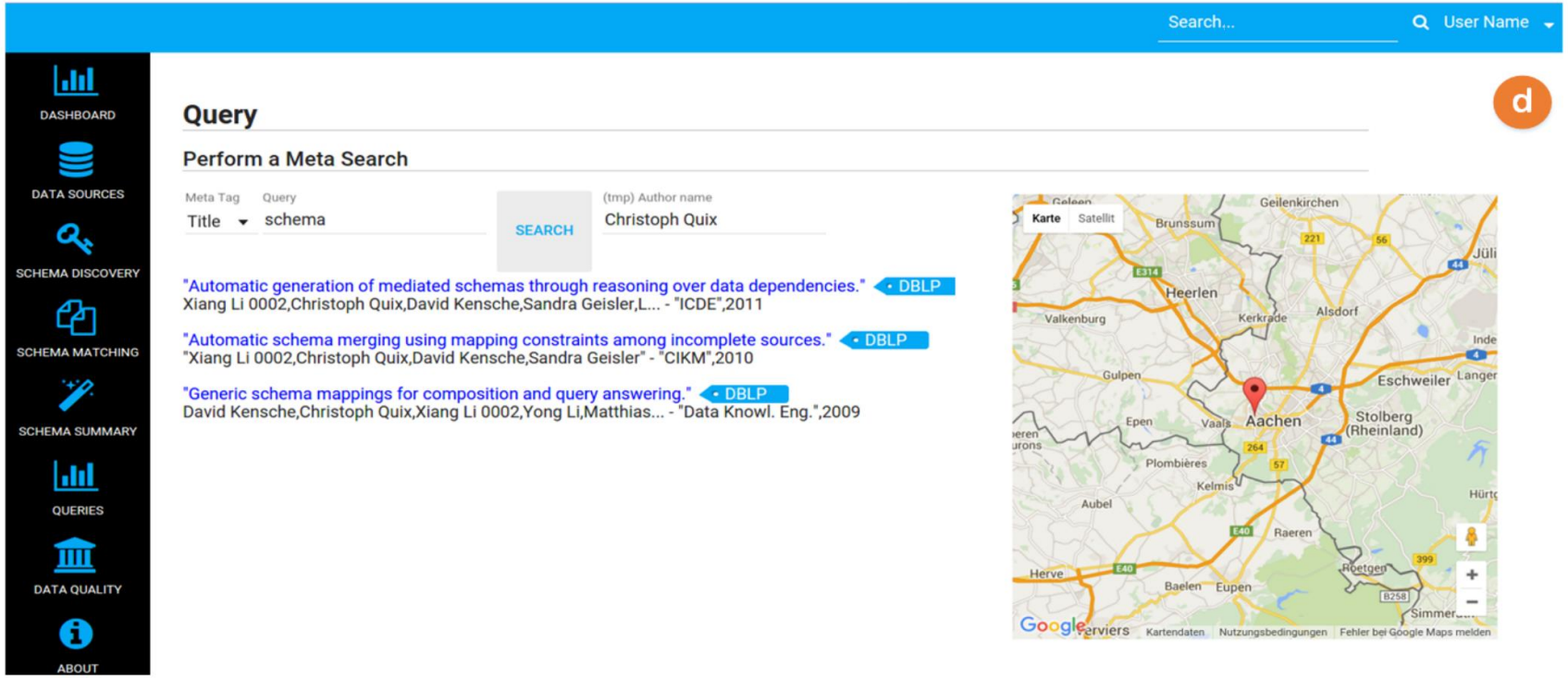
Hai R, Geisler S, Quix C. Constance: An intelligent data lake system[C]//Proceedings of the 2016 international conference on management of data. 2016: 2097-2100.(SIGMOD)

数据湖应用









Hai R, Geisler S, Quix C. Constance: An intelligent data lake system[C]//Proceedings of the 2016 international conference on management of data. 2016: 2097-2100.

数据仓库与数据湖对比分析

对比项	数据仓库	数据湖
存储规模	大量	海量
存储内容	清洗后的数据、整合提炼后的模型	原始数据、处理后的数据
数据类型	结构化数据	任意格式数据
数据应用	企业通用的批量加工	海量任意格式的批量加工、流式处理、分析探查
开发模式	要求提给专业团队，集中式开发	搭建自主平台，各相关方众包式开发（中台）
使用场景	报表、指标等稳态加工	快速、灵活的分析探查

3.2 第一代数据湖

➢ 基于Hadoop HDFS的数据湖

- HDFS（Hadoop Distributed File System）是最早的数据湖存储方案，主要用于**大数据存储与批处理**
- 数据通常存储为 Parquet、ORC、Avro 等开放文件格式，并由 Hive、Spark 进行计算分析。
- 特点：
  - ✅ 适用于**批处理和离线分析**
  - ✅ 兼容 Hadoop 生态，如 Hive、Spark、Presto
  - ❌ **实时查询性能较差**，不适用于低延迟查询
  - ❌ **数据治理较复杂**，缺乏统一的元数据管理
- 典型应用
  - **企业数据仓库**：存储长期业务数据，供 Hive/Spark 进行离线分析。
  - **日志分析**：存储 Web 日志、IoT 设备数据，进行大规模分析。

3.2 第一代数据湖

基于AWS S3的数据湖

- Amazon S3 是云端对象存储，可以作为数据湖的存储层，配合 AWS Glue、Athena、Redshift Spectrum 进行查询和分析。
- 特点：
  - ✅ **弹性存储**，适用于海量数据
  - ✅ 支持 SQL **查询**（Athena、Presto），兼容开源大数据工具
  - ❌ 读取性能受限（S3 本质是对象存储，非高性能数据库）
  - ❌ **事务支持较弱**，数据一致性依赖 Glue/S3 机制
- 典型应用
  - **机器学习**：S3 存储训练数据，供 SageMaker 进行建模。

3.2 第一代数据湖

➢ Azure Data Lake Storage (ADLS)

- 微软提供的云数据湖，支持与 Azure Synapse Analytics、Databricks、Power BI 等工具集成
- 特点：
  - ✅ **深度集成 Azure 生态**，适合使用 Azure 云服务的企业
  - ✅ 支持 **结构化 & 非结构化数据**，可直接进行分析
  - ❌ 主要依赖 Azure **生态**，迁移到其他平台较困难
- 典型应用
  - **企业大数据分析**：存储海量数据，并在 Azure Synapse 进行分析。
  - **流式数据处理**：结合 Event Hub/Kafka 进行实时数据存储。

3.2 第一代数据湖

- Google Cloud Storage (GCS) + BigQuery
- Google 提供的**对象存储数据湖**，结合 BigQuery 可用于**大规模 SQL 分析**。
- 特点：
  - ✅ **支持 Serverless 查询（BigQuery）**，不需要复杂运维
  - ✅ 与 AI/ML **平台（Vertex AI）** 集成，适合 AI 数据处理
  - ❌ **计算存储分离**，适合分析但不适合高并发事务处理
- 典型应用
  - **广告分析**：存储广告投放数据，使用 BigQuery 进行分析。
  - **零售行业**：存储商品销售数据，支持 BI 统计和趋势预测。

3.2 第一代数据湖

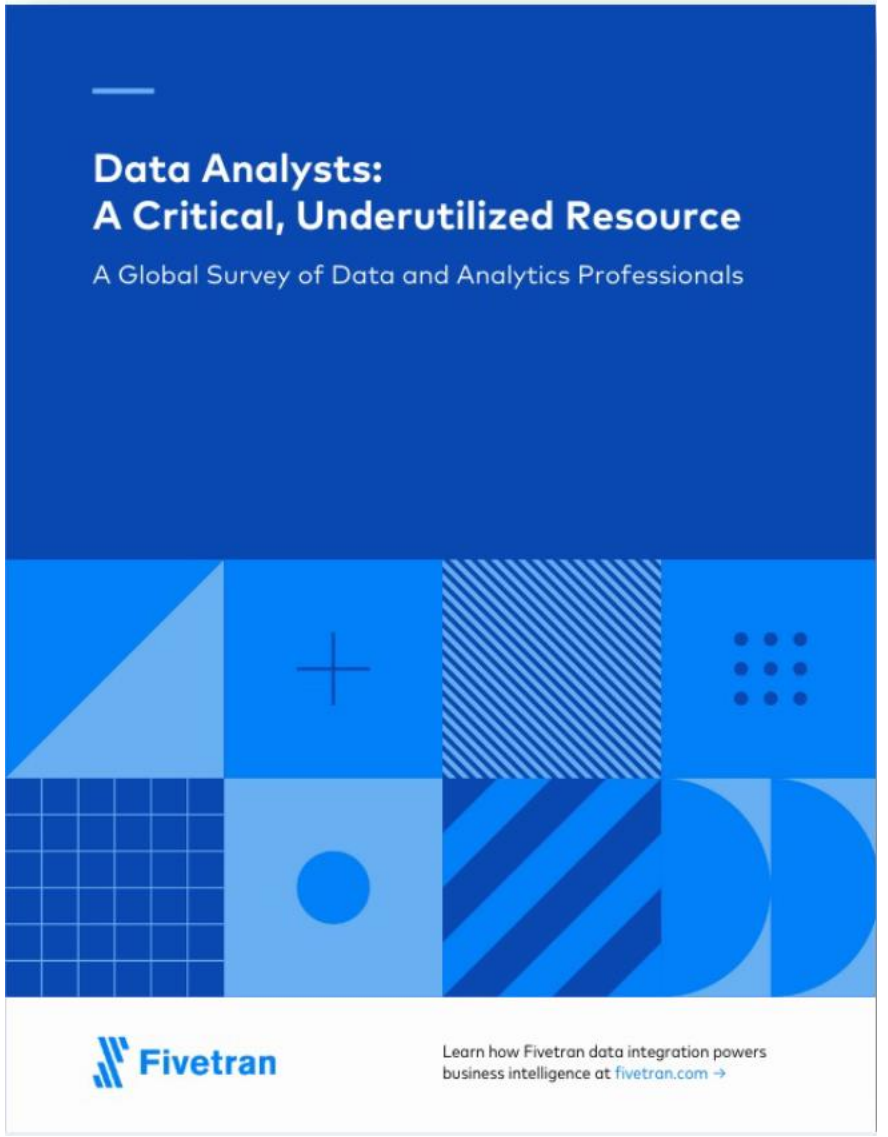
- 传统RDBMS扩展的数据湖
- 一些企业选择将 **传统数据库（如 PostgreSQL、MySQL）** 扩展成数据湖，例如：
  - PostgreSQL + Citus（扩展存储能力）
  - MySQL + HeatWave（支持大规模数据分析）
  - Oracle Data Lake（Oracle 数据库与对象存储结合）
- 这类方案适用于**需要兼顾事务处理和分析的业务**，但扩展性不如 Hadoop 或云数据湖。

目录

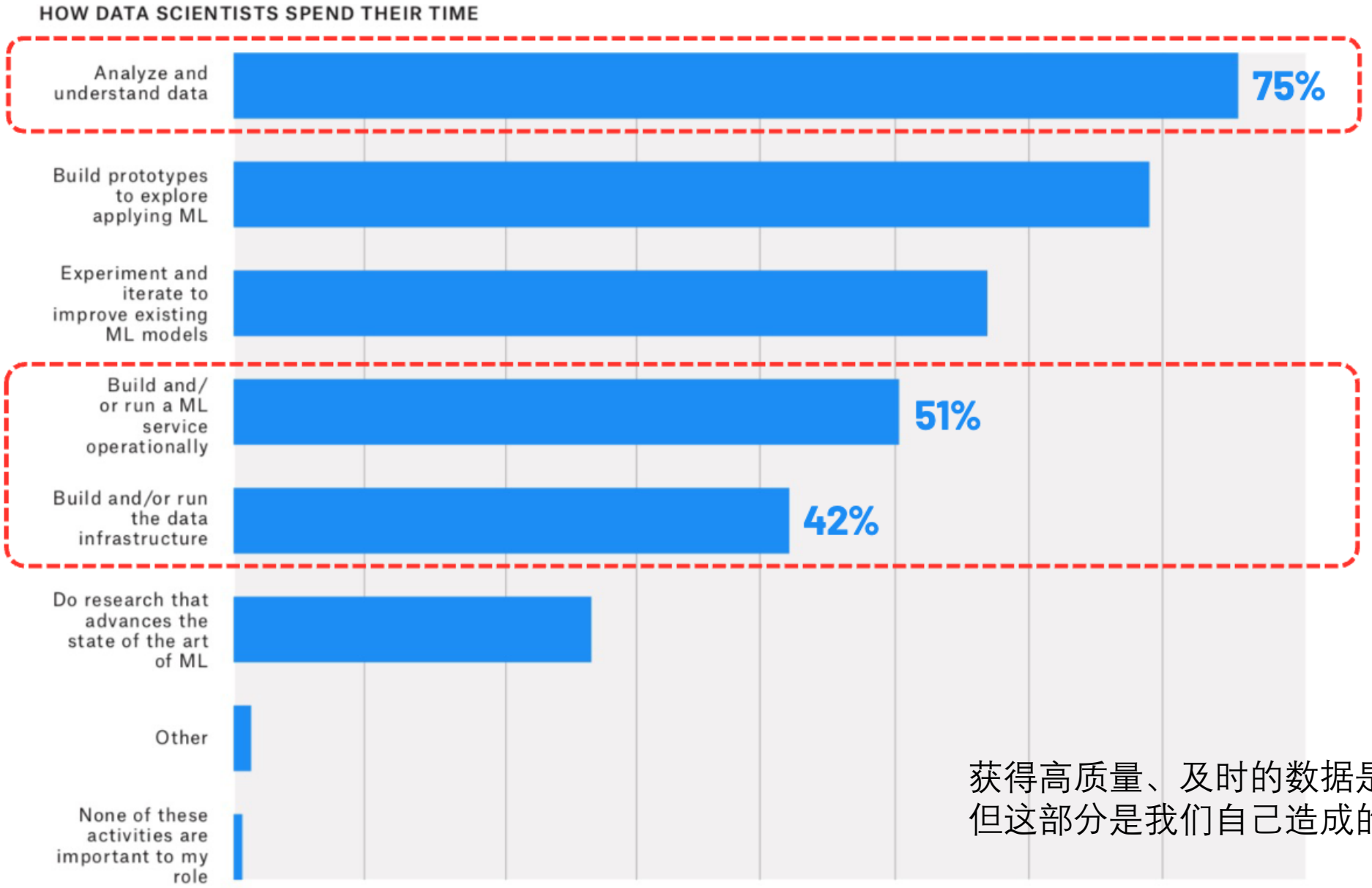
- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

4.1 当前数据面临的挑战

- 数据质量和时效性（Staleness）
- 60% 的受访者表示数据质量是首要挑战
- 86% 的分析师不得不使用过时的数据，其中 41% 的分析师使用 >2 个月的数据
- 90% 的受访者经常有不可靠的数据来源



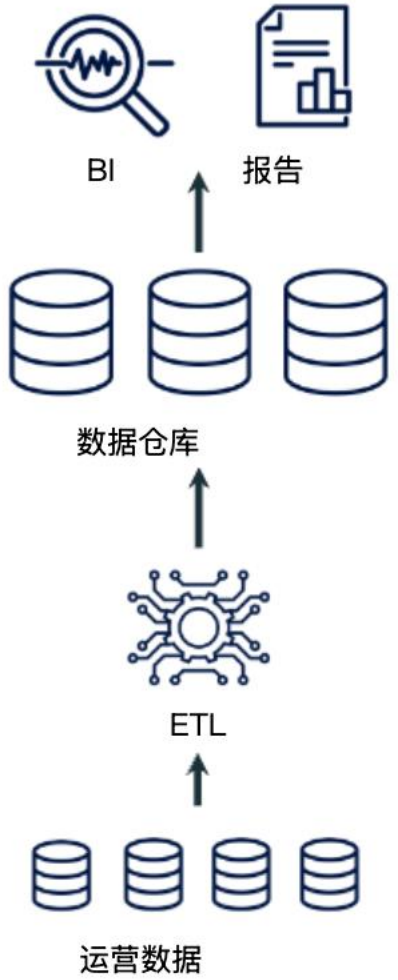
4.1 当前数据面临的挑战



获得高质量、及时的数据是很困难的.....
但这部分是我们自己造成的问题!

4.2数据管理的演变

- 1980 年代：数据仓库
  - 直接从运营数据库系统获取 ETL 数据
  - 专为SQL分析和BI构建：模式、索引、缓存等
  - 强大的管理功能，例如 ACID 事务
- 2010 年代：数据仓库的新问题
  - 无法支持快速增长的非结构化和半结构化数据：时间序列、日志、图像、文档等
  - 存储大型数据集的成本高
  - 不支持数据科学和机器学习



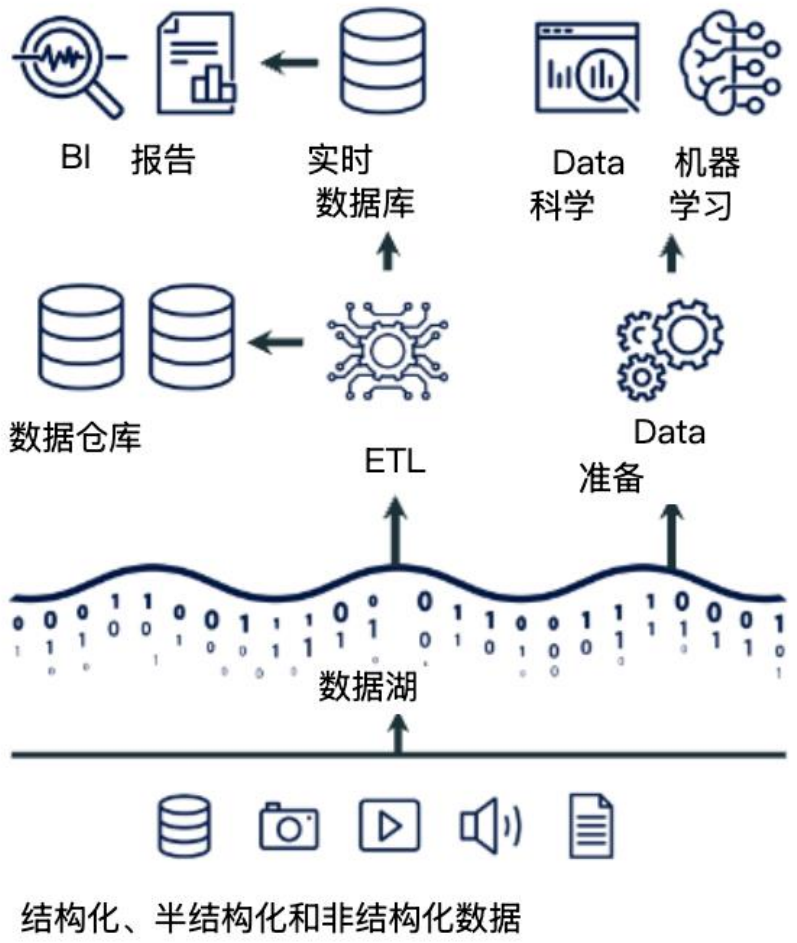
4.2数据管理的演变

2010 年代：数据仓库的新问题

- 无法支持快速增长的非结构化和半结构化数据：时间序列、日志、图像、文档等
- 存储大型数据集的成本高
- 不支持数据科学和机器学习

2010 年代：数据湖

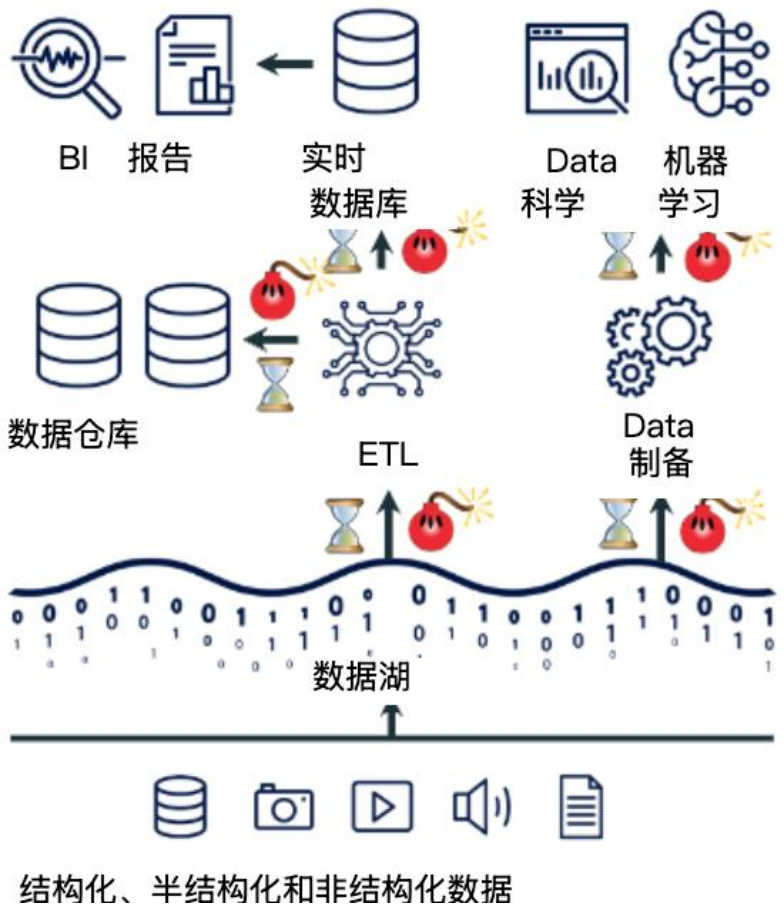
- 用于保存所有原始数据（例如 Amazon S3、HDFS）的低成本存储（S3 不频繁套餐 12 USD/TB/月）
- ETL 作业将特定数据加载到仓库中，可能用于进一步的 ELT
- 由于文件格式开放，可在 ML 库（例如 TensorFlow）中直接读取



4.2数据管理的演变

2010 年代：数据湖

- 用于保存所有原始数据（例如 Amazon S3、HDFS）的低成本存储（S3 不频繁套餐 12 USD/TB/月）
- ETL 作业将特定数据加载到仓库中，可能用于进一步的 ELT
- 由于文件格式开放，可在 ML 库（例如 TensorFlow）中直接读取



当今数据湖的问题

- 数据可靠性受到影响：
  - 具有不同语义、SQL 方言等的多个存储系统
  - 可能出错的额外 ETL 步骤
- 时效性受到影响：
  - 在数据仓库中提供数据之前，额外的 ETL 步骤

4.2数据管理的演变

总结

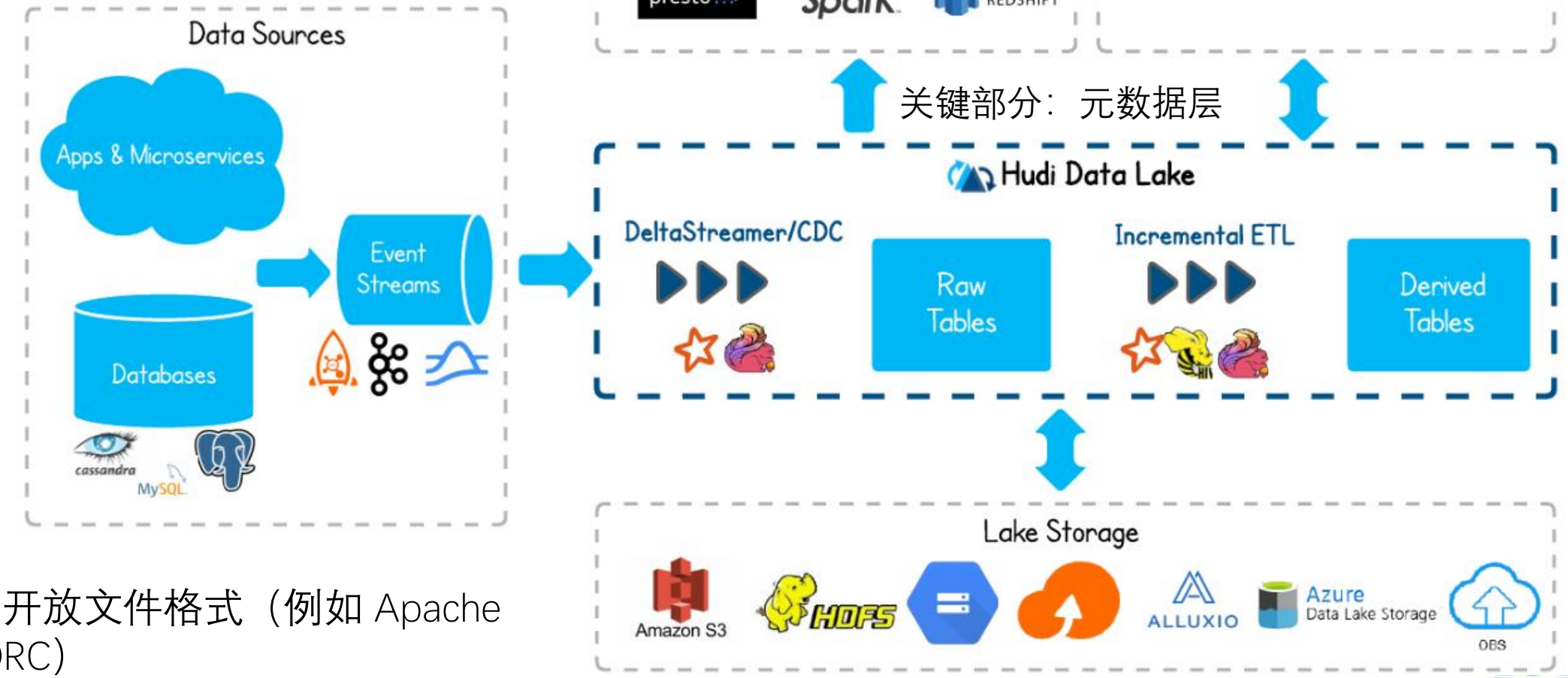
- 现代数据架构中的至少一些问题是由于不必要的系统复杂性造成的
- 我们希望为大型历史数据提供低成本存储，但我们为此设计了单独的存储系统（数据湖）
- 现在我们需要一直跨系统同步数据!
- 如果我们不需要拥有所有这些不同的数据系统，那会怎样?

4.3 湖仓一体架构

直接在 Data Lake Storage 上提供数据仓库功能的新技术

在顶部添加管理和性能功能（事务、数据版本控制、索引等）

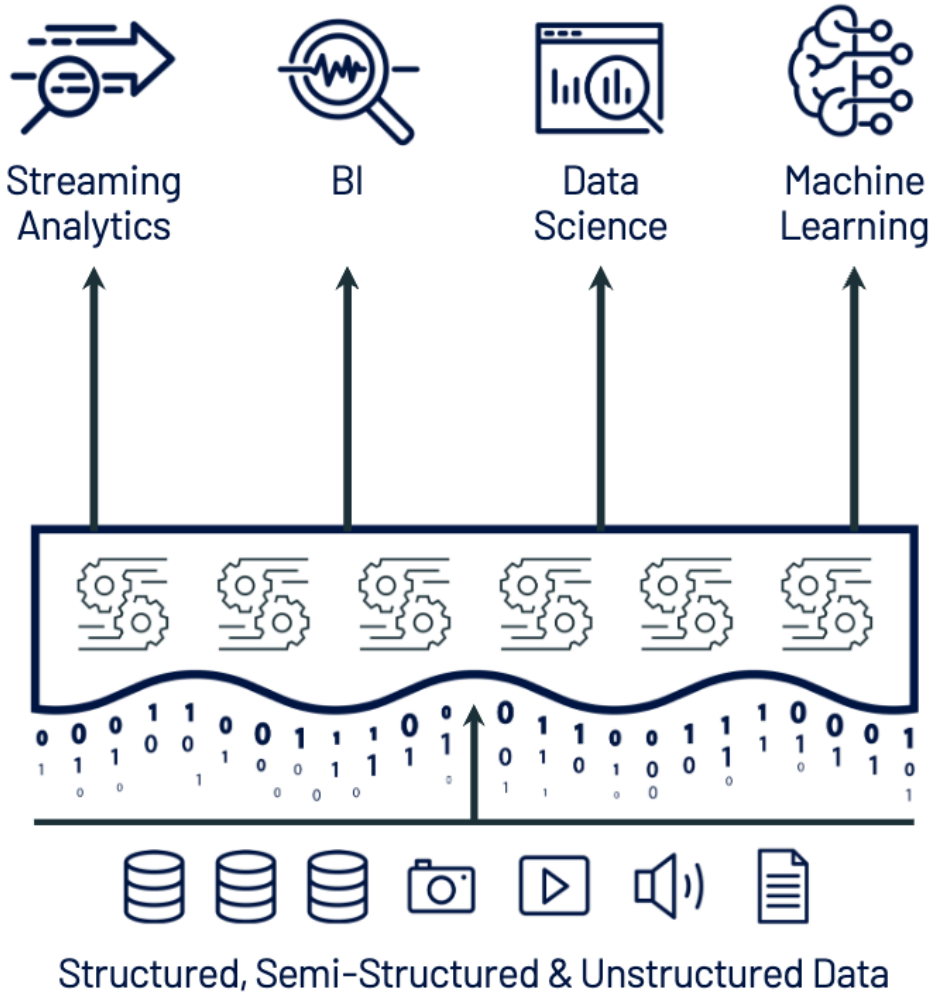
消除其他数据系统，例如消息队列



保留现有的开放文件格式（例如 Apache Parquet、ORC）

4.3 湖仓一体架构

Lakehouse愿景



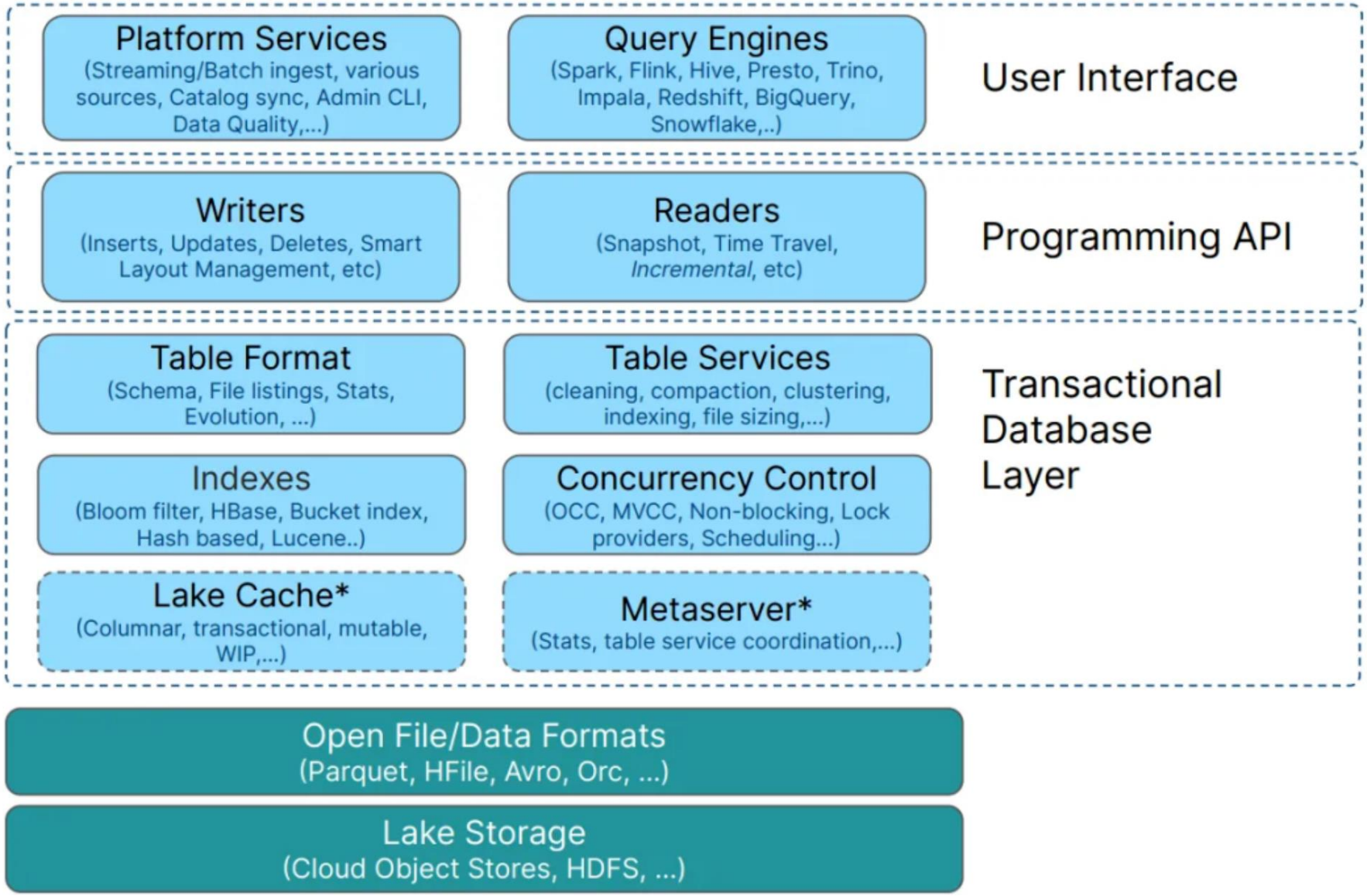
适用于各种用例的单一平台

管理功能（事务、版本控制等）

适用于所有数据的 Data Lake Storage

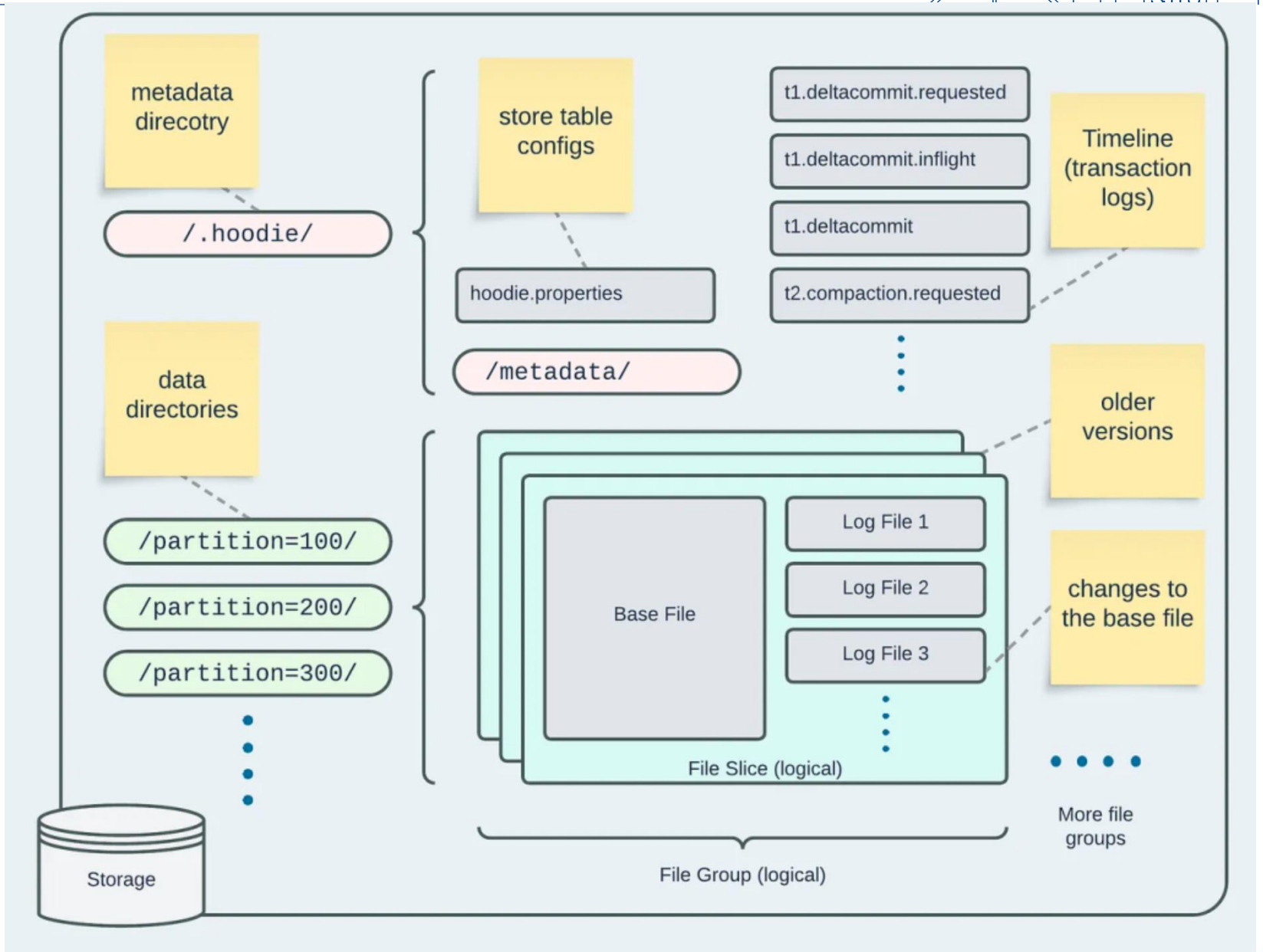
4.3 湖仓一体架构

- Apache Hudi架构



4.3 湖仓一体架构

- Apache Hudi 数据模型



4.3 湖仓一体架构

Lakehouse关键技术

- 1.存储层：添加事务、版本控制等
- 2.新的查询引擎设计：在数据湖存储系统和文件格式上实现出色的 SQL 性能
- 3.优化数据科学和机器学习的访问

4.3 湖仓一体架构

- Lakehouse关键技术
  - 1.数据湖的元数据层：添加事务、版本控制等
  - 2.新的查询引擎设计：在数据湖存储系统和文件格式上实现出色的 SQL 性能
  - 3.优化数据科学和机器学习的访问

4.4 元数据层

- 数据湖通常只是文件的集合
- 元数据层会跟踪哪些文件是表的一部分，以实现更丰富的管理功能，例如事务
  - 客户端仍然可以高速访问底层文件
- 在多个系统中实现：



Databricks



Netflix



ACID

将元数据保存在对象存储本身中

将元数据保存在数据库中

4.4 元数据层

数据模型

- 表 = 目录
- 按照Hive 分区命名规范
- 可以对文件进行分区（例如，按日期）。
- JSON 文件记录了 “actions” 数组：
  - 更改元数据
  - 添加/删除对象
  - 协议版本变更
  - 溯源
  - 特定应用程序的数据（例如，流系统可能用于提供“有效一次”语义的序列号）。
- 定期将日志（更改）检查点放入 Parquet 文件中。

DELTA LAKE

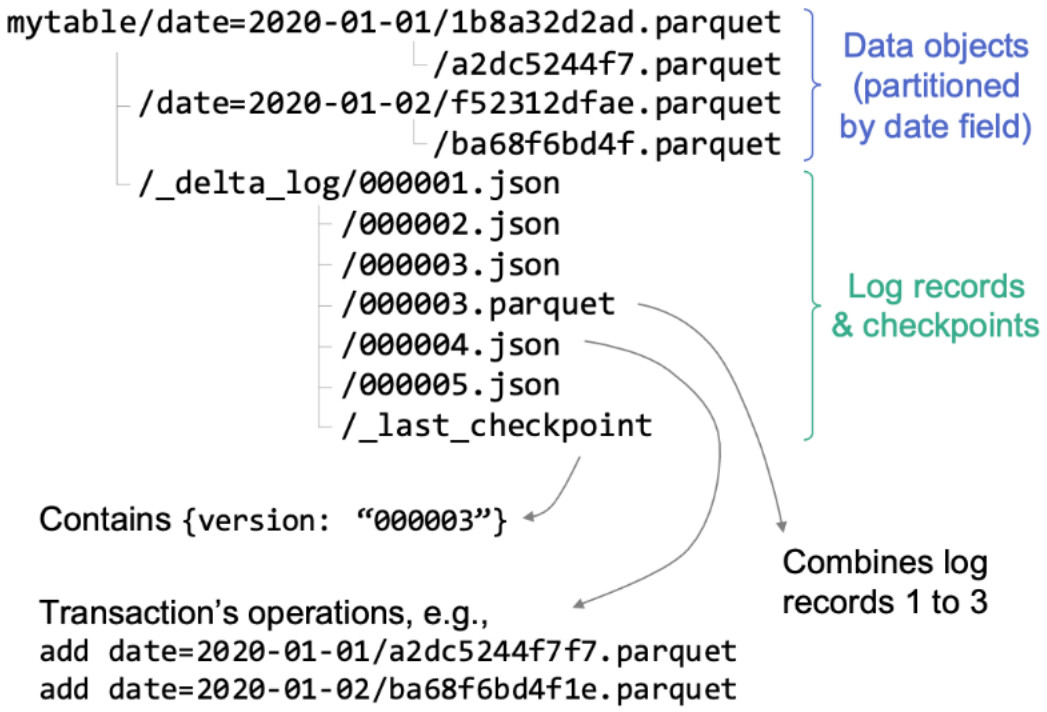
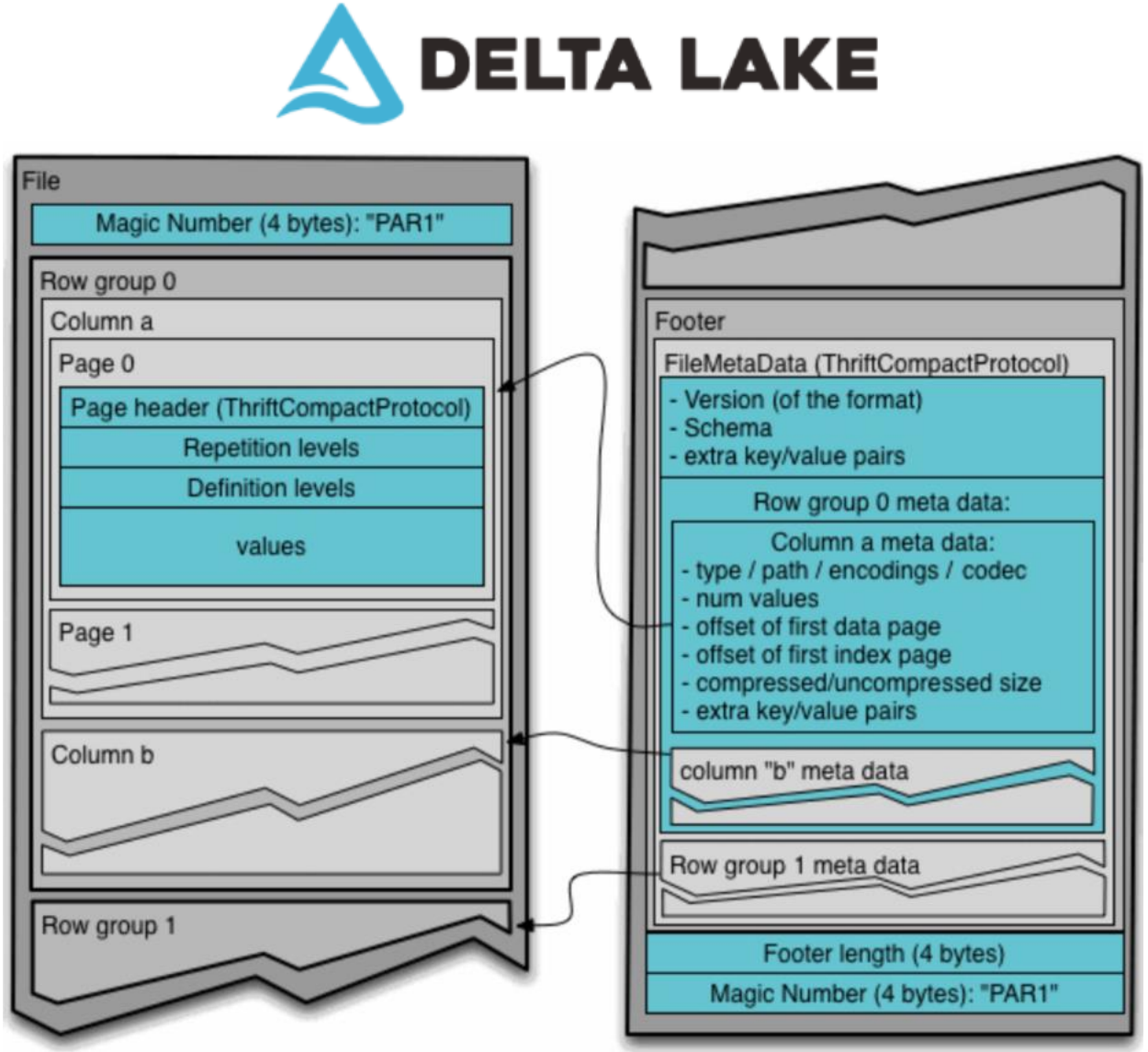


Figure 2: Objects stored in a sample Delta table.

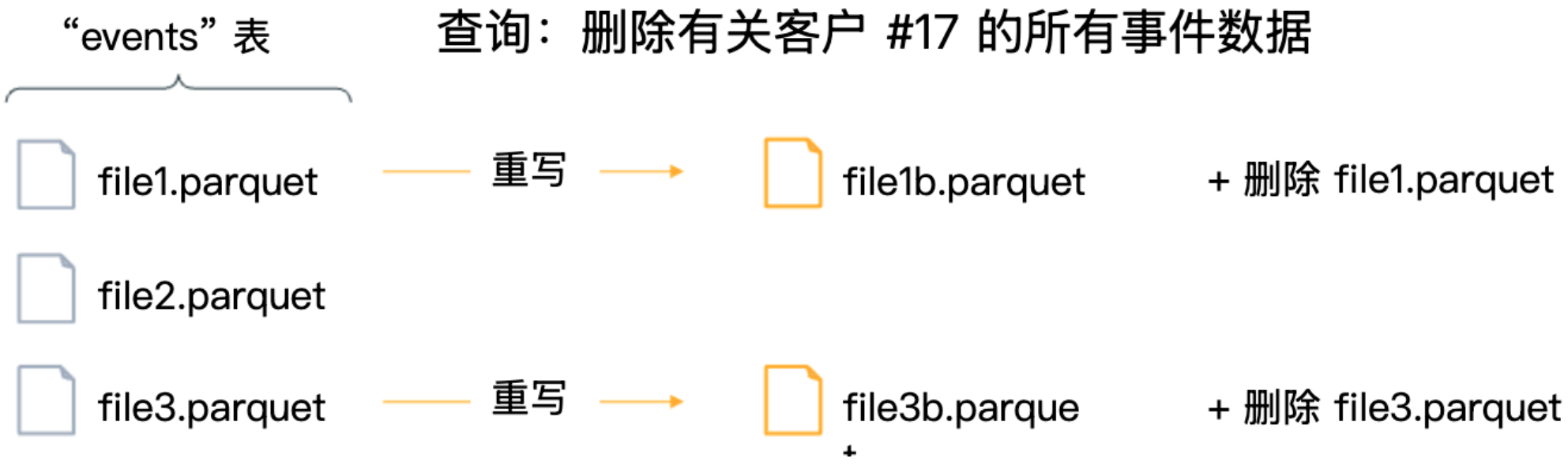
4.4 元数据层

- 数据模型
- Parquet file
  - 首先进行水平分区（行组）
  - 在每个行组中，使用列式组织。
  - 使用 Snappy、GZIP、LZO、BROTLI、LZ4、ZSTD LZ4_RAW 进行压缩
  - 每个行组的统计信息（min-max 和 bloom 筛选器）。
  - 元数据在footer中，因此文件是自描述和自包含的。



4.4 元数据层

例子：基本数据湖

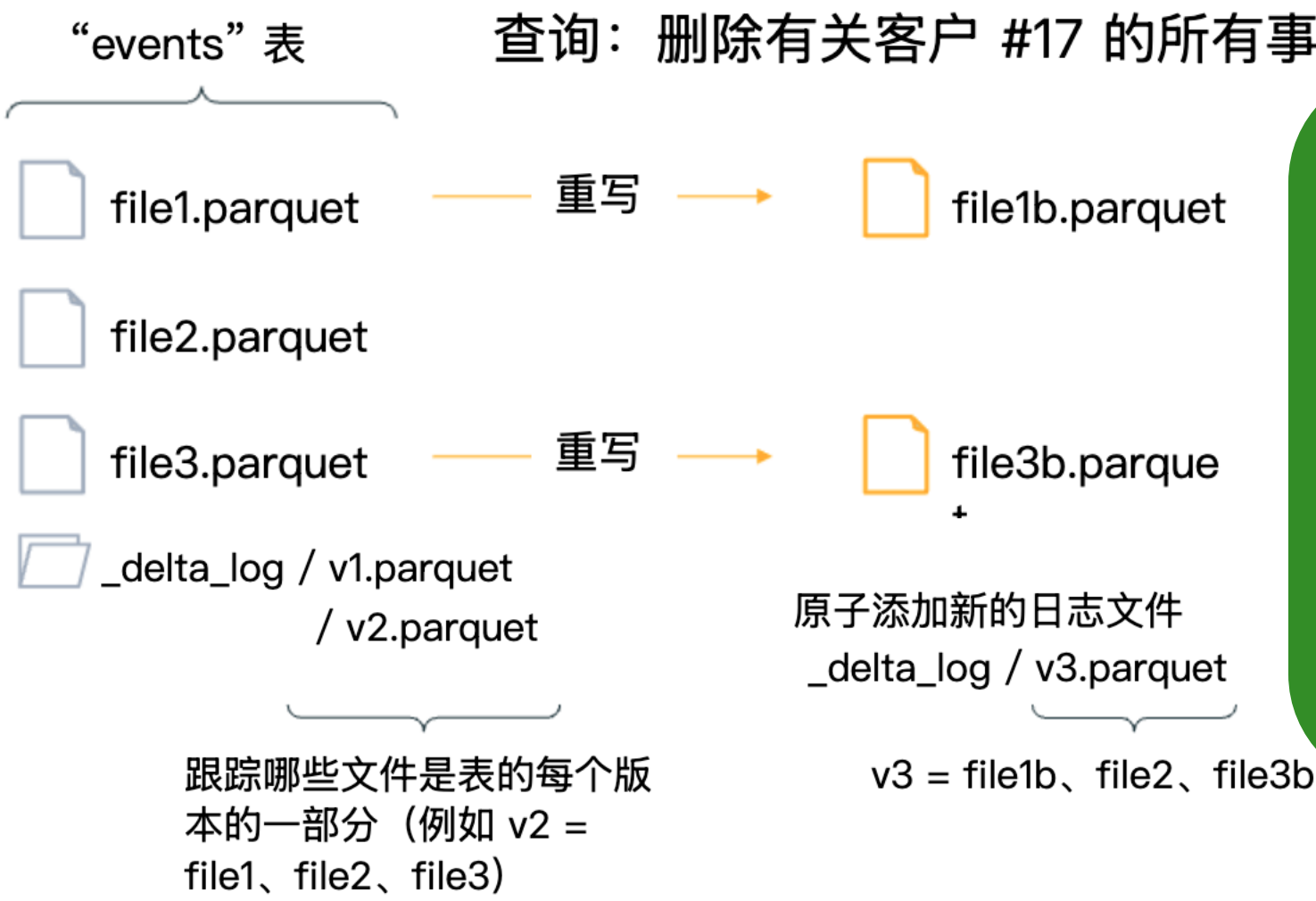


问题：如果查询在执行删除操作时读取表，该怎么办？

问题：如果执行删除的查询中途失败，该怎么办？

4.4 元数据层

- 例子：DELTA LAKE



客户端始终读取一致的表版本!

- 如果客户端读取 v2 的日志，则会看到 file1、file2、file3（无删除）
- 如果客户端读取 v3 的日志，它将看到 file1b、file2、file3b（全部已删除）

4.4 元数据层

其他管理功能

- 时间旅行到旧表格版本

```
SELECT * FROM my_table
TIMESTAMP AS OF “2020-05-01”
```

- 通过分叉日志实现零拷贝 CLONE

```
CREATE TABLE my_table_dev
SHALLOW CLONE my_table
```

- 描述历史

```
1 DESCRIBE HISTORY flightdelays
```

▸ (1) Spark Jobs

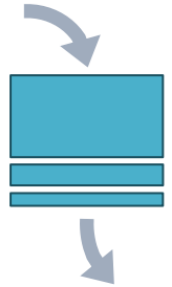
version	timestamp	userId	userName	operation	notebook
7	2019-10-08T16:47:22	101543	...@databricks.com	MERGE	{"notebookId":"25"}
6	2019-10-08T16:44:16	101543	...@databricks.com	MERGE	{"notebookId":"25"}
5	2019-10-06T19:26:53	101543	...@databricks.com	UPDATE	{"notebookId":"25"}

- 插入、更新、删除和合并

4.4 元数据层

➢ 其他管理功能

- 流式 I/O：将表视为更改流，以消除对 Kafka 等消息总线的需求



```
spark.readStream
     .format("delta")
     .table("events")
```

- Schema执行和演变

```
CREATE TABLE orders (
  product_id INTEGER NOT NULL,
  quantity INTEGER CHECK(quantity > 0),
  list_price DECIMAL CHECK(list_price > 0),
  discount_price DECIMAL
    CHECK(discount_price > 0 AND
          discount_price <= list_price)
);
```

- 对数据质量的期望

4.4 元数据层

成效

- 数千家公司每天使用 EB 级数据
- 在 3 年内从 0 增长到 ~50% 的 Databricks 工作负载
- 最大的部署：EB 级表和数千个用户



4.5 存储优化设计

- Lakehouse关键技术
  - 1.数据湖的元数据层：添加事务、版本控制等
  - 2.新的查询引擎设计：在数据湖存储系统和文件格式上实现出色的 SQL 性能
  - 3.优化数据科学和机器学习的访问

- 存储优化挑战
  - 大多数数据仓库对数据存储系统和查询引擎具有完全控制权，因此它们一起设计
  - Lakehouse 的关键思想是以开放存储格式（例如 Parquet）存储数据，以便从许多系统直接访问
  - 我们如何使用这些标准的开放格式获得出色的性能？

4.5 存储优化设计

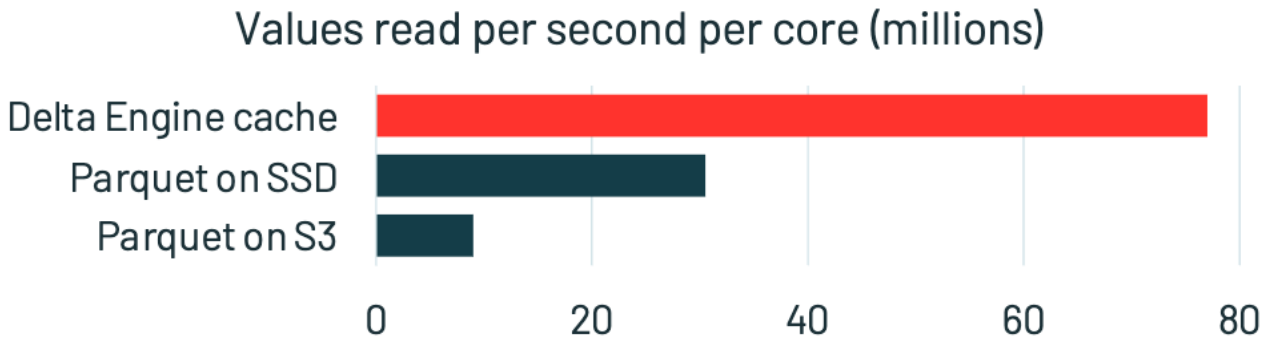
- 即使使用固定的、可直接访问的存储格式，四项优化也可以实现出色的 SQL 性能：
  - 缓存热数据，可能采用不同的格式
  - 辅助数据结构，如统计信息和索引
  - 数据布局优化以最大限度地减少 I/O
  - 适用于现代 CPU 的向量化执行引擎

新的查询引擎（如 Databricks Delta 引擎）使用这些想法

4.5 存储优化设计

优化方案 1：缓存

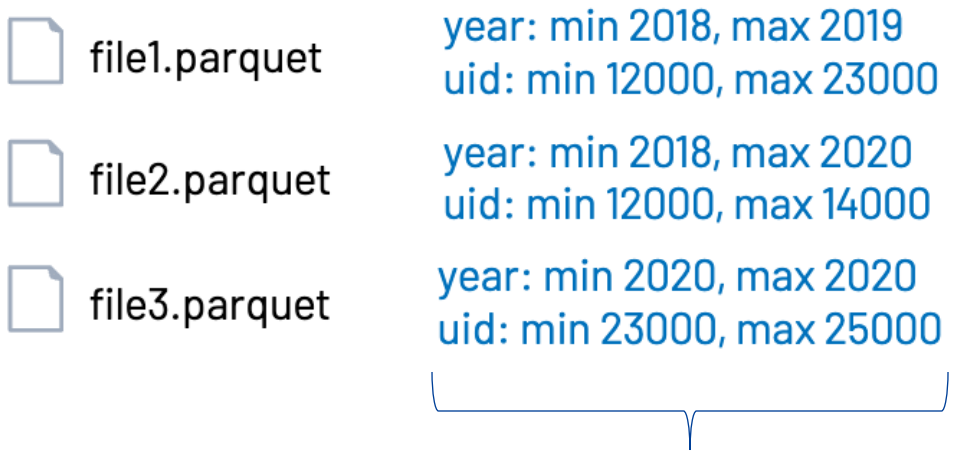
- 大多数查询工作负载都集中在 “热” 数据上
  - 数据仓库使用 SSD 和内存缓存来提高性能
- 如果我们有一个元数据层（如 Delta Lake）来正确维护缓存，则相同的技术在 Lakehouse 中也适用
  - 缓存甚至可以以更快的格式保存数据（例如解压缩）
- 示例：Databricks Delta Engine 中的 SSD 缓存



4.5 存储优化设计

➢ 优化方案 2：辅助数据结构

- 即使基础数据位于 Parquet 中，我们也可以构建许多其他数据结构来加速查询并以事务方式维护它们
- 示例1：用于数据跳过的 Parquet 文件的最小/最大统计信息

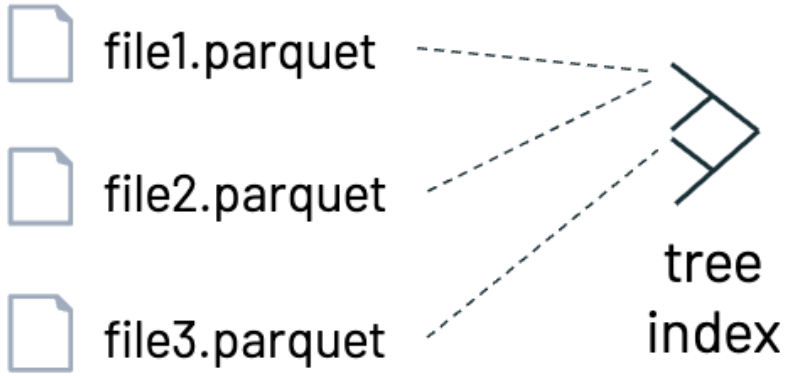


Query:
```
SELECT * FROM events
WHERE year=2020 AND uid=24000
```

4.5 存储优化设计

优化方案 2：辅助数据结构

- 即使基础数据位于 Parquet 中，我们也可以构建许多其他数据结构来加速查询并以事务方式维护它们
- 示例2：Parquet 文件的索引




```
Query: SELECT * FROM events
       WHERE type = “DELETE_ACCOUNT”
```

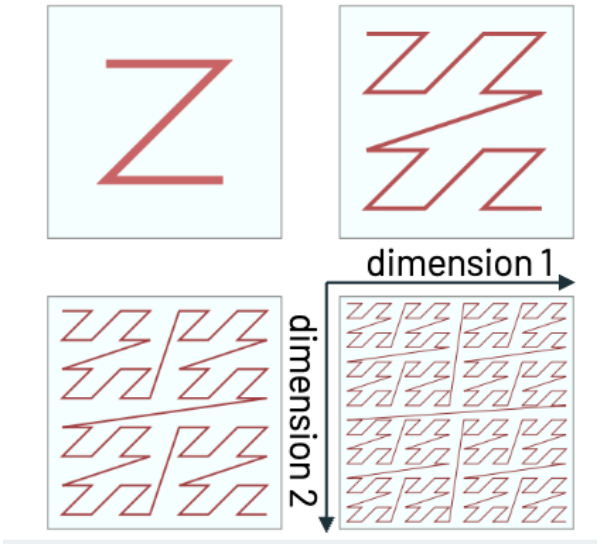
4.5 存储优化设计

- 优化方案 3：数据布局
  - 查询执行时间主要取决于访问的数据量
  - 即使使用 Parquet 等固定存储格式，我们也可以优化表内的数据布局以减少执行时间
- 示例1：对表进行排序以便快速访问

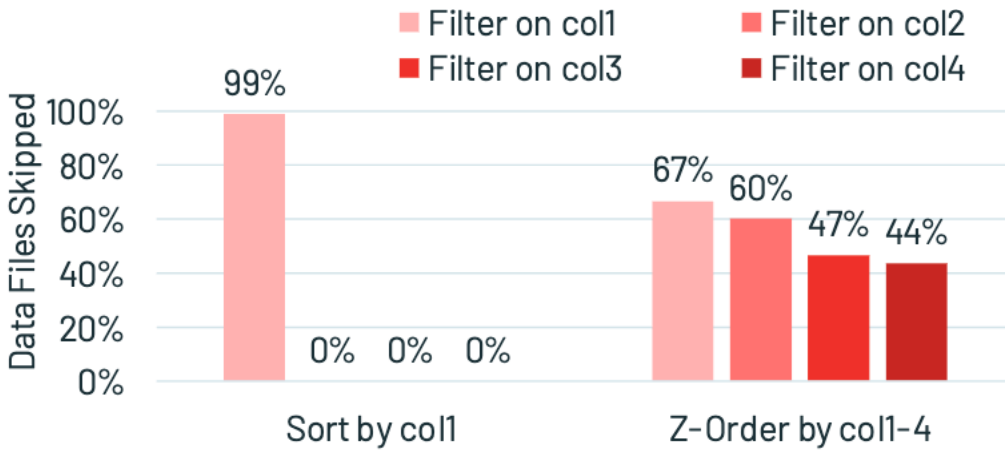


4.5 存储优化设计

- 优化方案 3：数据布局
  - 查询执行时间主要取决于访问的数据量
  - 即使使用 Parquet 等固定存储格式，我们也可以优化表内的数据布局以减少执行时间
- 示例2：用于多维访问的 Z-ordering



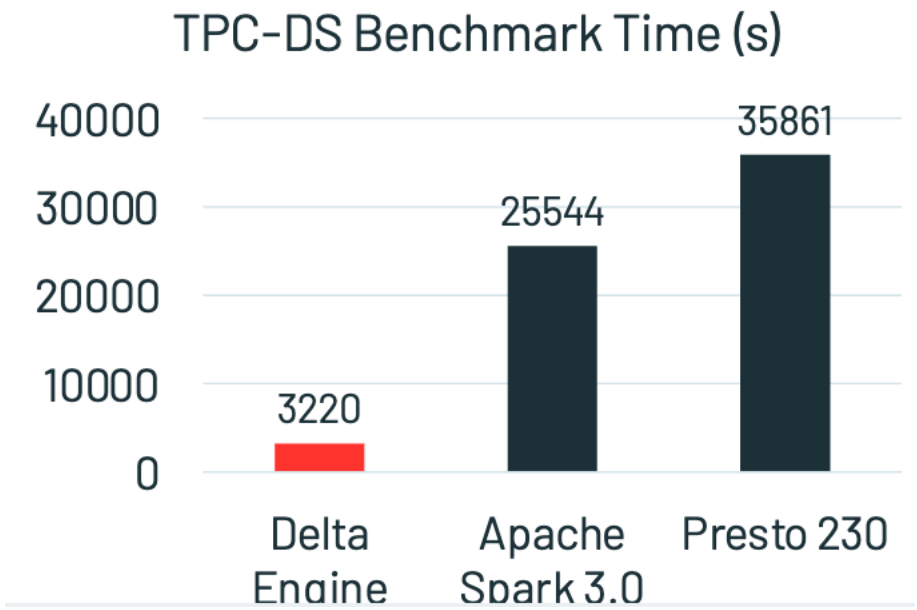

```
OPTIMIZE events ZORDER BY (eventType, generateTime)
```



4.5 存储优化设计

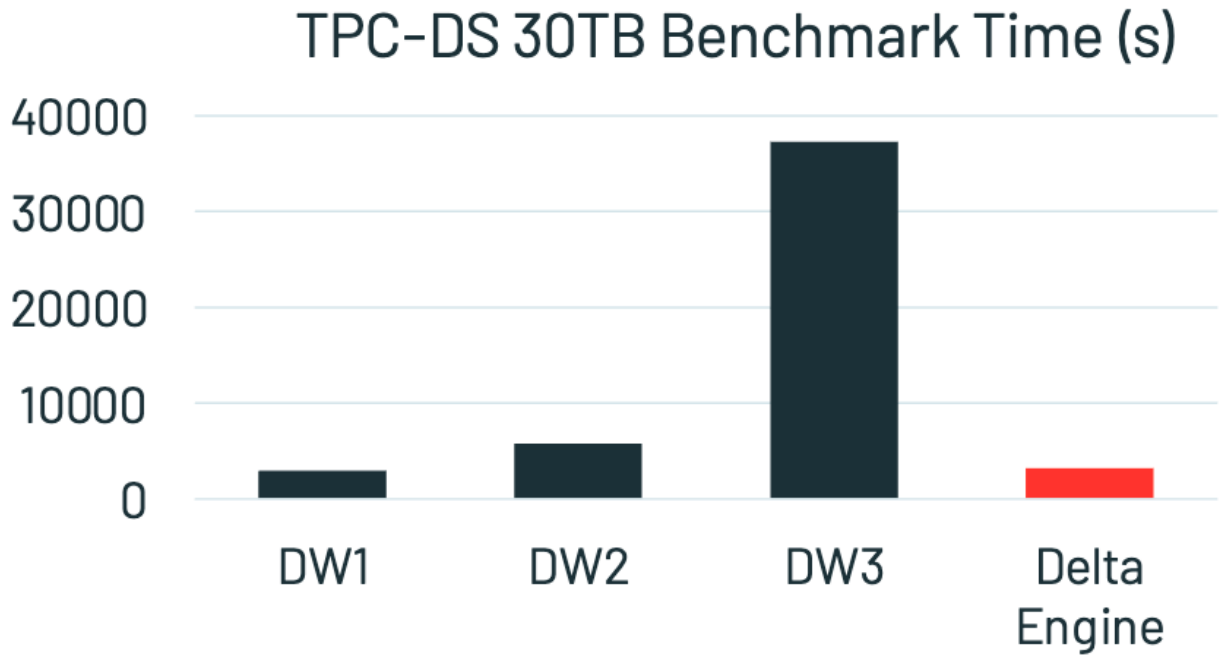
优化方案 4：向量化执行

- 现代数据仓库通过在现代 CPU（例如 AVX512）上使用矢量 （SIMD） 指令来优化 CPU 时间
- 其中许多优化也可以应用于 Parquet
- Databricks Delta 引擎：比基于 Java 的引擎快 ~10 倍



4.5 存储优化设计

- 由于（1） 大多数读取来自缓存，（2） I/O 成本是非缓存数据的关键因素，以及 （3） CPU 时间可以通过 SIMD 进行优化...
- Lakehouse 可以提供与 DW 相似的性能！



4.6 Lakehouse for ML

- Lakehouse关键技术
  - 1.数据湖的元数据层：添加事务、版本控制等
  - 2.新的查询引擎设计：在数据湖存储系统和文件格式上实现出色的 SQL 性能
  - 3.优化数据科学和机器学习的访问

4.6 Lakehouse for ML

数仓上的ML

- 与 SQL 工作负载不同，ML 工作负载需要使用非 SQL 代码（例如 TensorFlow、XGBoost 等）处理大量数据
- 基于 JDBC/ODBC 接口的 SQL 对于大规模操作来说太慢了
- 将数据导出到数据湖？→增加了第三个 ETL 步骤和更多的过时性！
- 在 DW 和 Lake 中维护生产数据集？→更复杂

湖仓一体上的ML

- 直接访问数据文件，而不会使 SQL 前端过载（例如，只需运行 GPU 集群即可对 S3 数据进行深度学习）
  - ML 框架已经支持读取 Parquet！
- 用于 ML 数据准备的新声明式 API 支持进一步优化

4.6 Lakehouse for ML

示例：Spark 的声明式 DataFrame API

- 用户使用 Python、R 或 Java 编写 DataFrame 代码

```
users = spark.table(“users”)
buyers = users[users.kind == “buyer”]
train_set = buyers[“start_date”, “zip”, “quantity”]
                  .fillna(0)
```

4.6 Lakehouse for ML

➢ 示例：Spark 的声明式 DataFrame API

- 用户使用 Python、R 或 Java 编写 DataFrame 代码

```
users = spark.table("users")
buyers = users[users.kind == "buyer"]
train_set = buyers["start_date", "zip", "quantity"]
                  .fillna(0)

...

model.fit(train_set)
```

databricks

延迟评估的查询计划
Lazily evaluated query plan

PROJECT(NULL → 0)
|
PROJECT(start_date, zip, ...)
|
SELECT(kind = "buyer")
|
users

DELTA LAKE

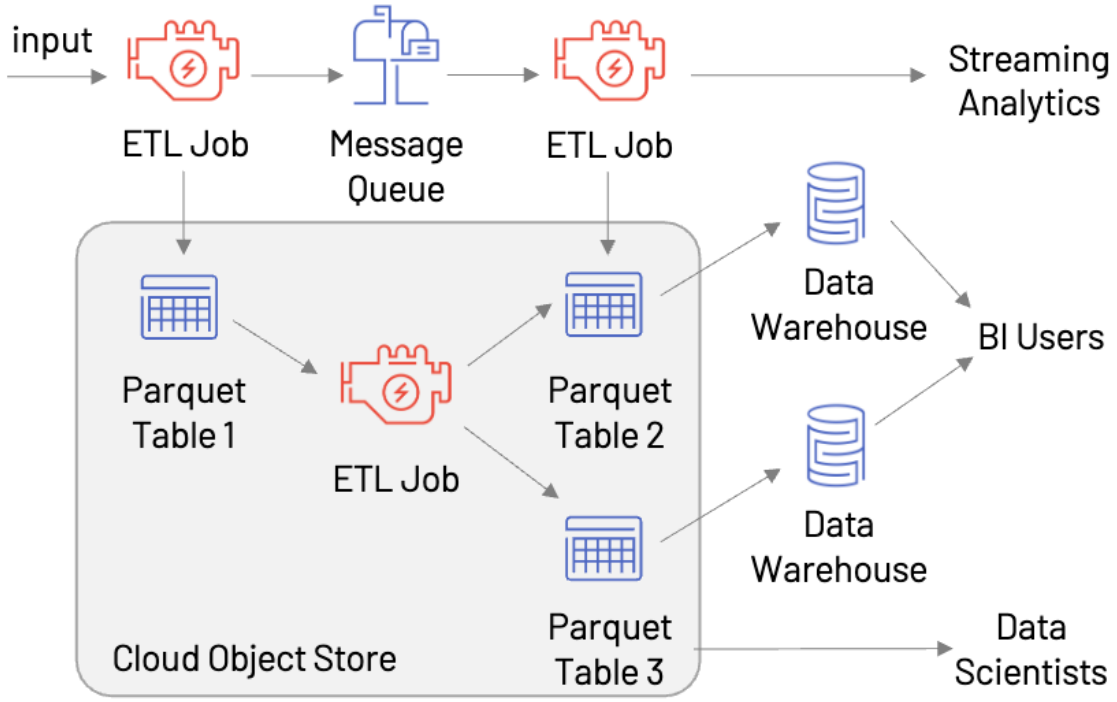
Optimized execution using
cache, statistics, index, etc

4.6 Lakehouse for ML

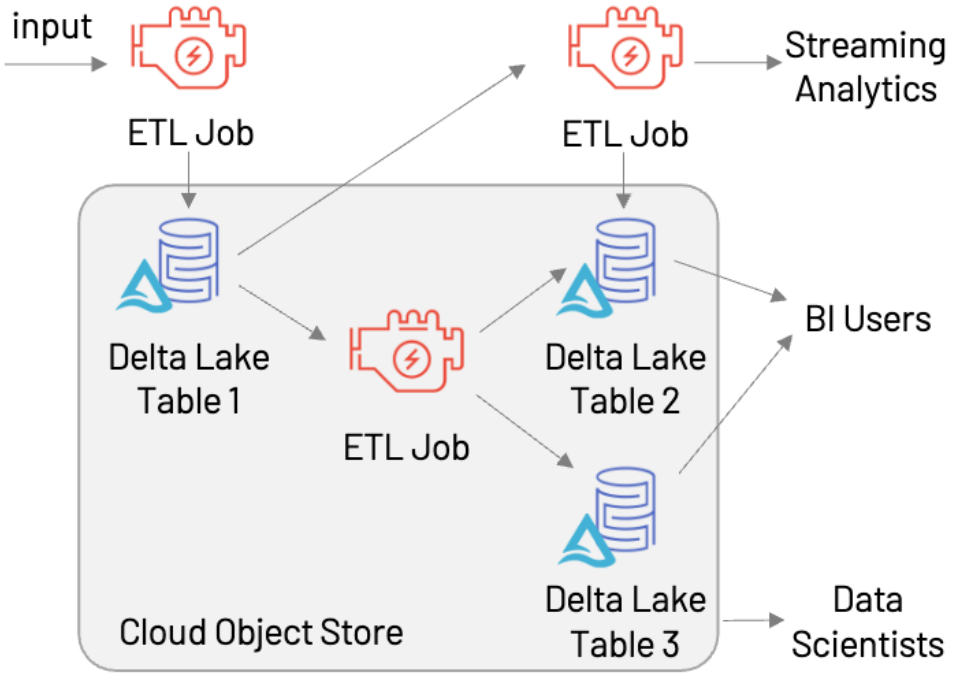
- Lakehouse 系统的管理功能也使 ML 更容易!
  - 使用时间旅行进行数据版本控制和可重现的实验
  - 使用事务可靠地更新表
  - 始终从流式 I/O 访问最新数据
- 示例：使用 Delta Lake 作为 ML“特征存储（Feature Store）”的组织

湖仓一体之前与之后

Typical Architecture with Many Data Systems



Lakehouse Architecture: All Data in Object Store



databricks

Fewer copies of the data, fewer ETL steps, no divergence & faster results!

目录

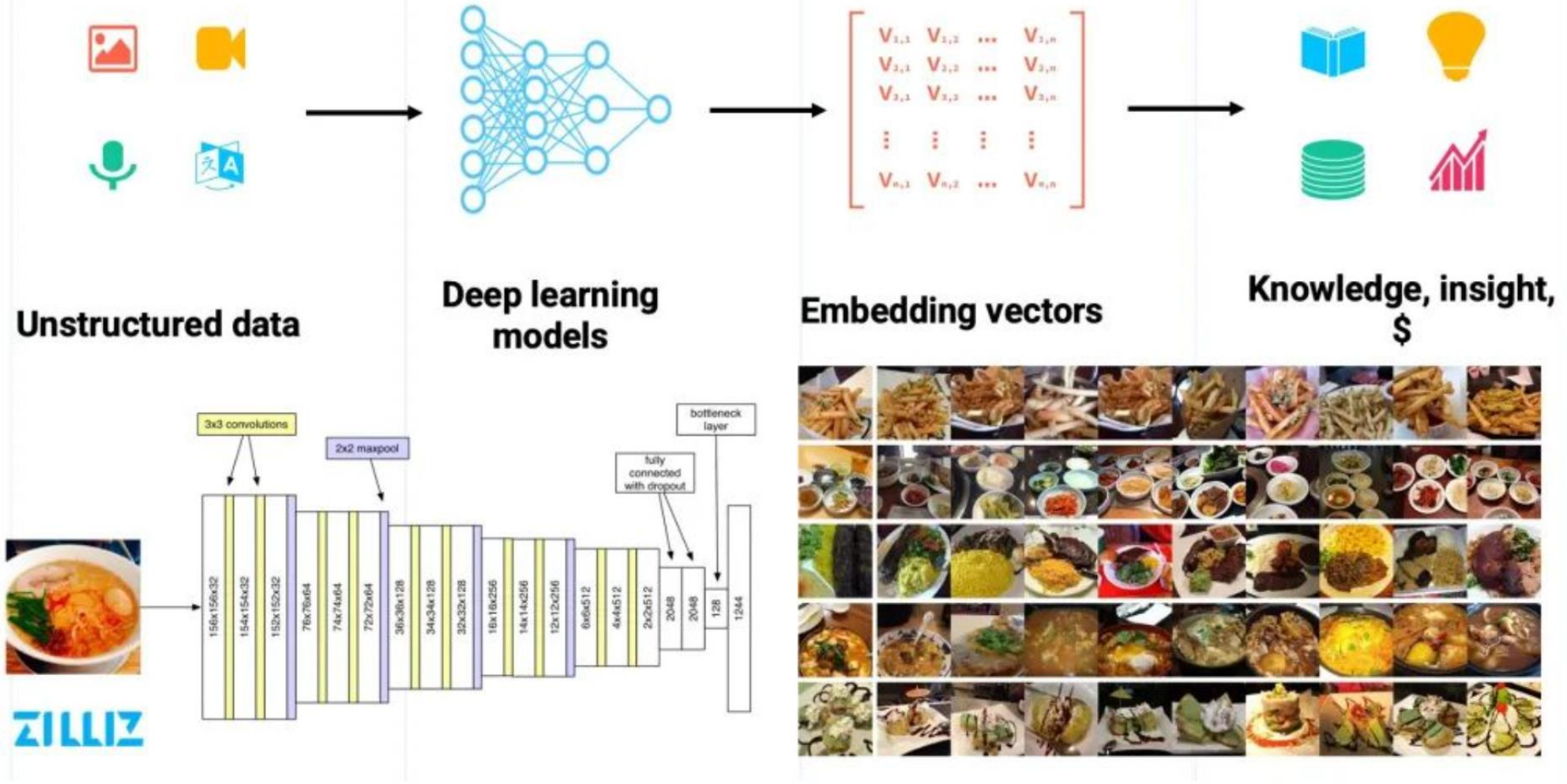
- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

5.1 为什么需要向量数据库

- 算据：随着互联网不断发展，电子邮件、论文、物联网传感数据、社交媒体照片、蛋白质分子结构等非结构化数据已经变得越来越普遍。
- 算力：至2024年9月，中国算力总规模达到246EFLOPS，算力应用项目超过1.3万个，在用算力中心机架总规模超过830万标准机架，位居世界前列
- 算法：ANN，CNN，Transformer，GPT，Deepseek
- 全方位改变软件开发格局

5.1 为什么需要向量数据库

非结构化数据处理流程



5.1 为什么需要向量数据库

向量与标量

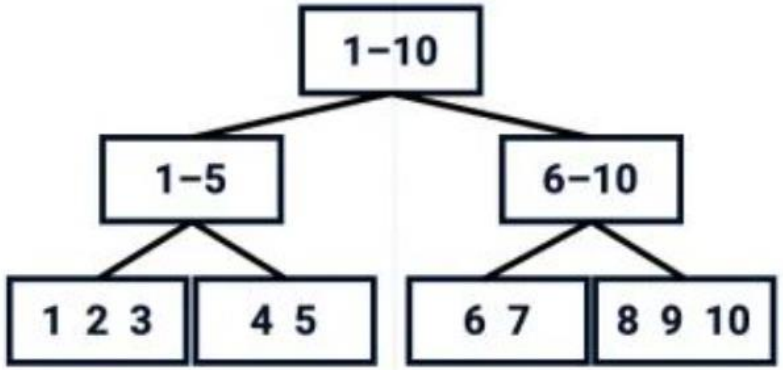
Numbers

Arithmetic operation

$+ - \times \div$

Number comparison

$a <=> b$



Operation

Organization

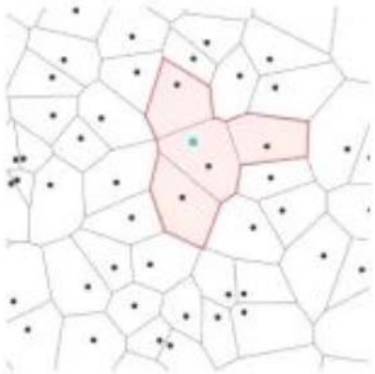
Vectors

Similarity (eg. Euclidean distance)

$$d(A,B)=\sqrt{\sum_{i=1}^{n}(a_i-b_i)^2}$$

Similarity comparison

$$\mathrm{TopK}(A)=\arg\min_{B\in\gamma}(d(A,B))$$

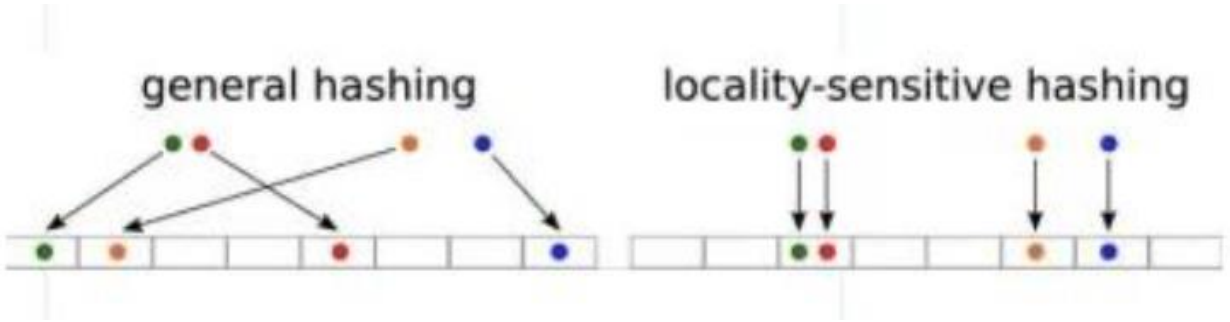


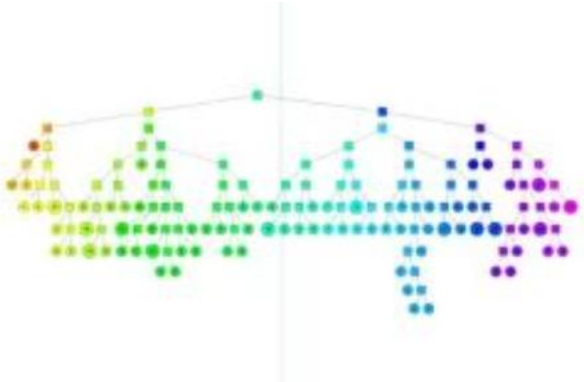


5.1 为什么需要向量数据库

常见的向量索引方式

- FLAT file：暴力搜索，这种方式是典型的牺牲性能和成本换取准确性，是唯一可以实现 100% 召回率的方式，同时可以较好地使用显卡等异构硬件加速
- Hash based：基于 locality sensitive hashing 将数据分到不同的哈希桶中。这种方式实现简单，性能较高，但是召回率不够理想。
- Tree based：KDTree 或者 BallTree，通过将高维空间进行分割，并在检索时通过剪枝来减少搜索的数据量，这种方式性能不高，尤其是在维度较高时性能不理想。

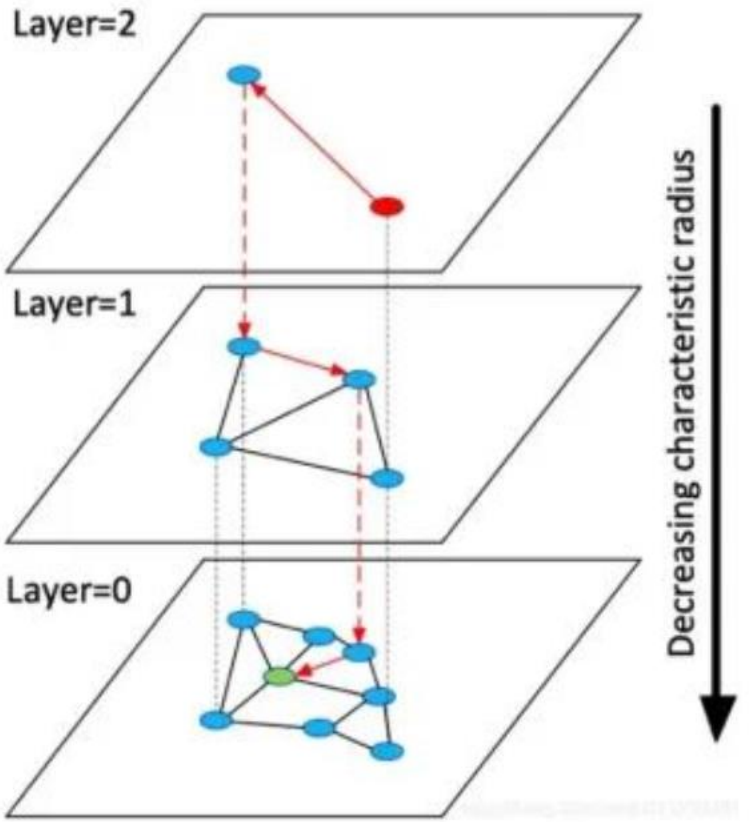




5.1 为什么需要向量数据库

➢ 常见的向量索引方式

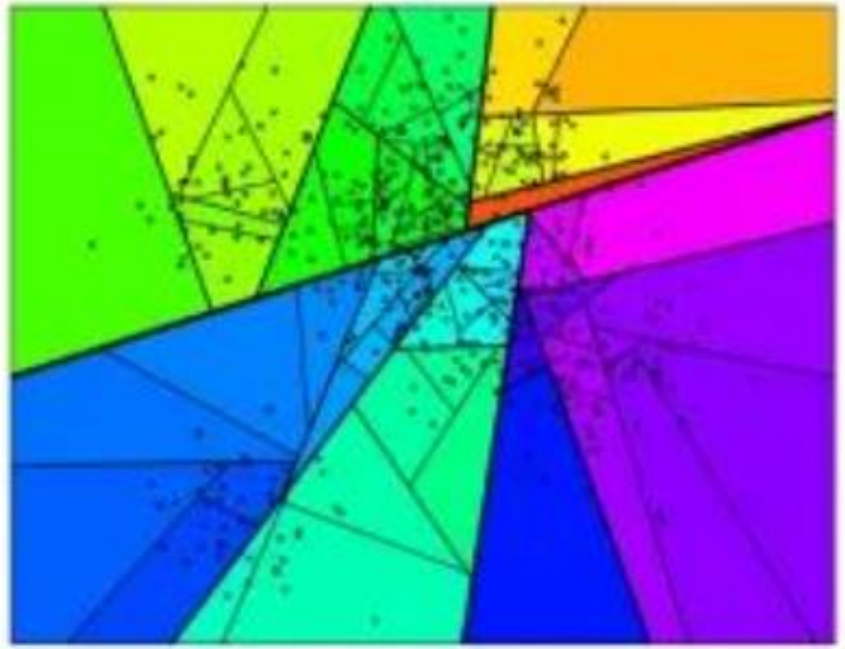
- 基于聚类的倒排：通过 k-means 算法找到数据的一组中心点，并在查询时利用查询向量和中心点距离选择部分桶进行查询。倒排这一类又拥有很多的变种，比如可以通过 PCA 将数据进行降维，进行标量量化，或者通过乘积量化 PQ 将数据降精度，这些都有助于减少系统的内存使用和单次数据计算量。



5.1 为什么需要向量数据库

常见的向量索引方式

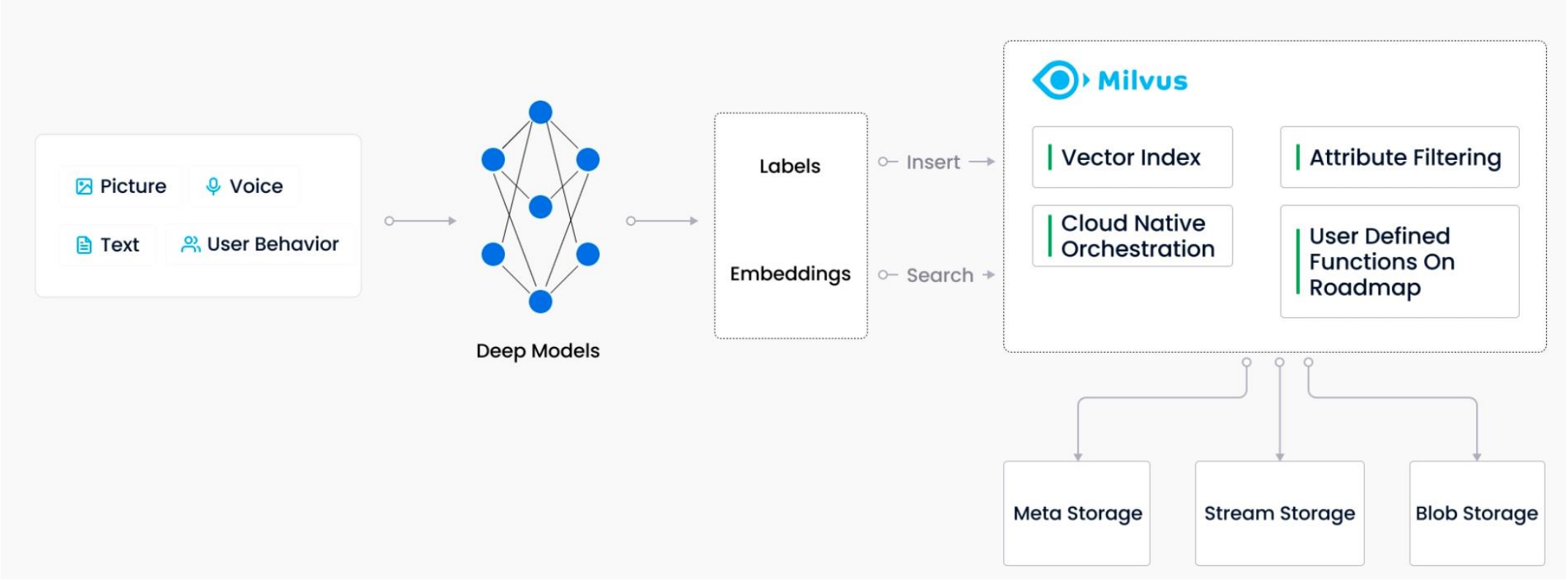
- NSW（Navigable Small World）：图是一种基于图存储的数据结构，这种索引基于一种朴素的假设，通过在构建图连接相邻的友点，然后在查询时不断寻找距离更近的节点实现局部最优。在 NSW 的基础上，HNSW（Navigable Small World）图借鉴了跳表的机制，通过层状结构构建了快速通道，提升了查询效率。



5.2 Milvus

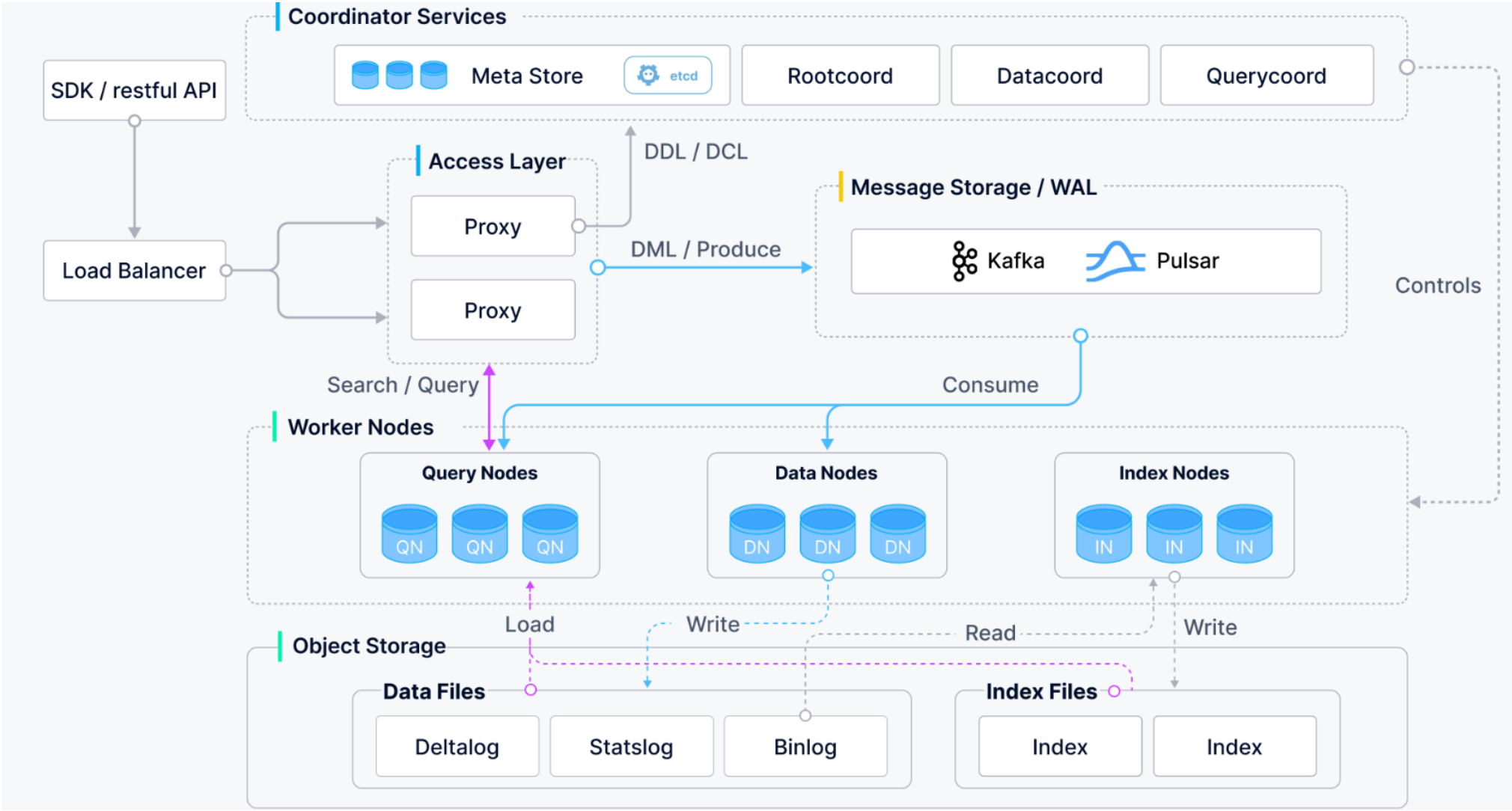
Milvus

- 一款云原生向量数据库，它具备高可用、高性能、易拓展的特点，用于海量向量数据的实时召回。



5.2 Milvus

- 存算分离
- 云原生



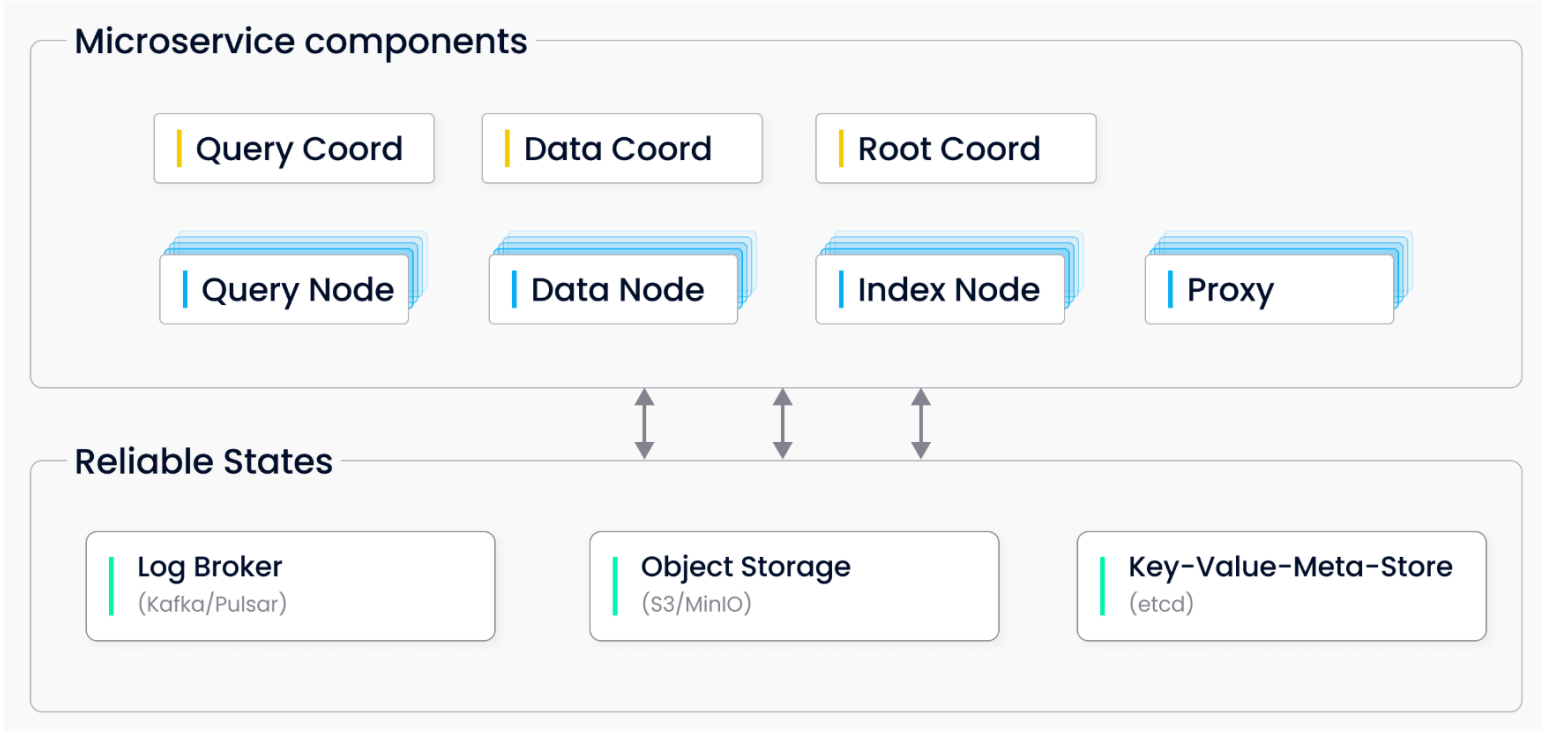
5.3 存储层

数据模型

- **Collection：**包含一组 entity，可以等价于关系型数据库系统（RDBMS）中的表。
- **Entity：**包含一组 field。field 与实际对象相对应。field 可以是代表对象属性的结构化数据，也可以是代表对象特征的向量。primary key 是用于指代一个 entity 的唯一值。
- **Field：**Entity 的组成部分。Field 可以是结构化数据，例如数字和字符串，也可以是向量。
- **Partition**
- **Segment：**Milvus 在数据插入时，通过合并数据自动创建的数据文件。一个 collection 可以包含多个 segment。一个 segment 可以包含多个 entity。在搜索中，Milvus 会搜索每个 segment，并返回合并后的结果。
- **Sharding**

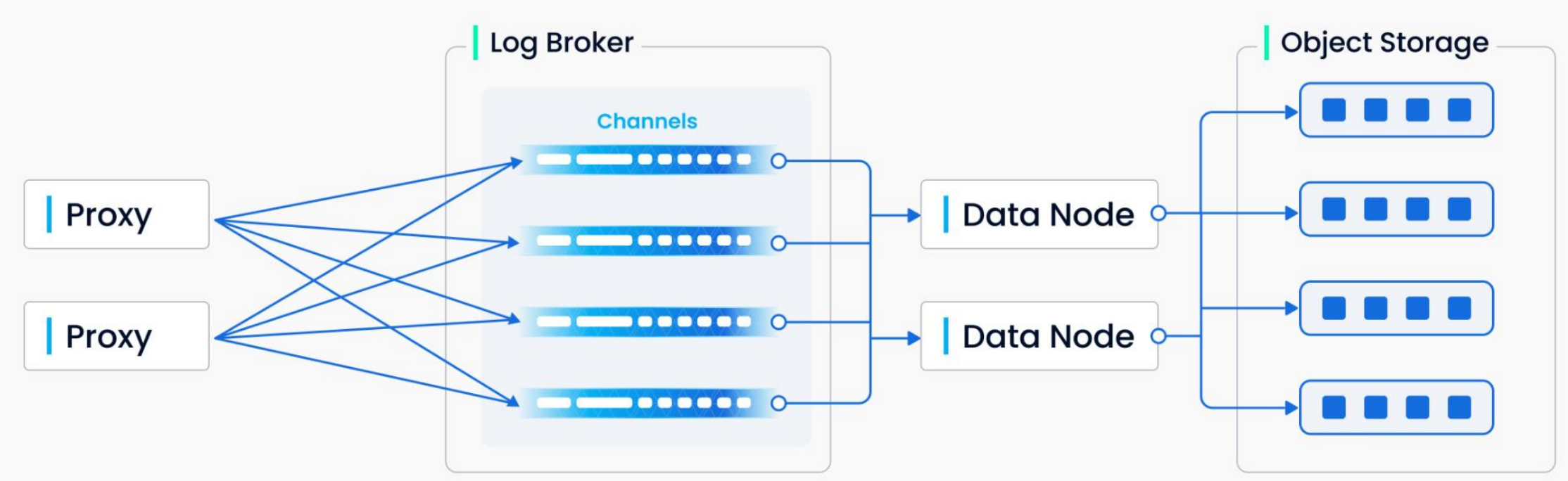
5.3 存储层

- 元存储：
  - 用于存储元数据，如集合模式和消息检查点。Milvus 使用 etcd 作为元存储，确保高可用性和强一致性。
- 对象存储：
  - 用于存储日志快照文件、索引文件和中间查询结果。Milvus 支持 MinIO、AWS S3 和 Azure Blob 作为对象存储。
- 日志代理：
  - 负责日志的持久化和发布-订阅服务。Milvus 集群使用 Pulsar 作为日志代理，单机版使用 RocksDB。



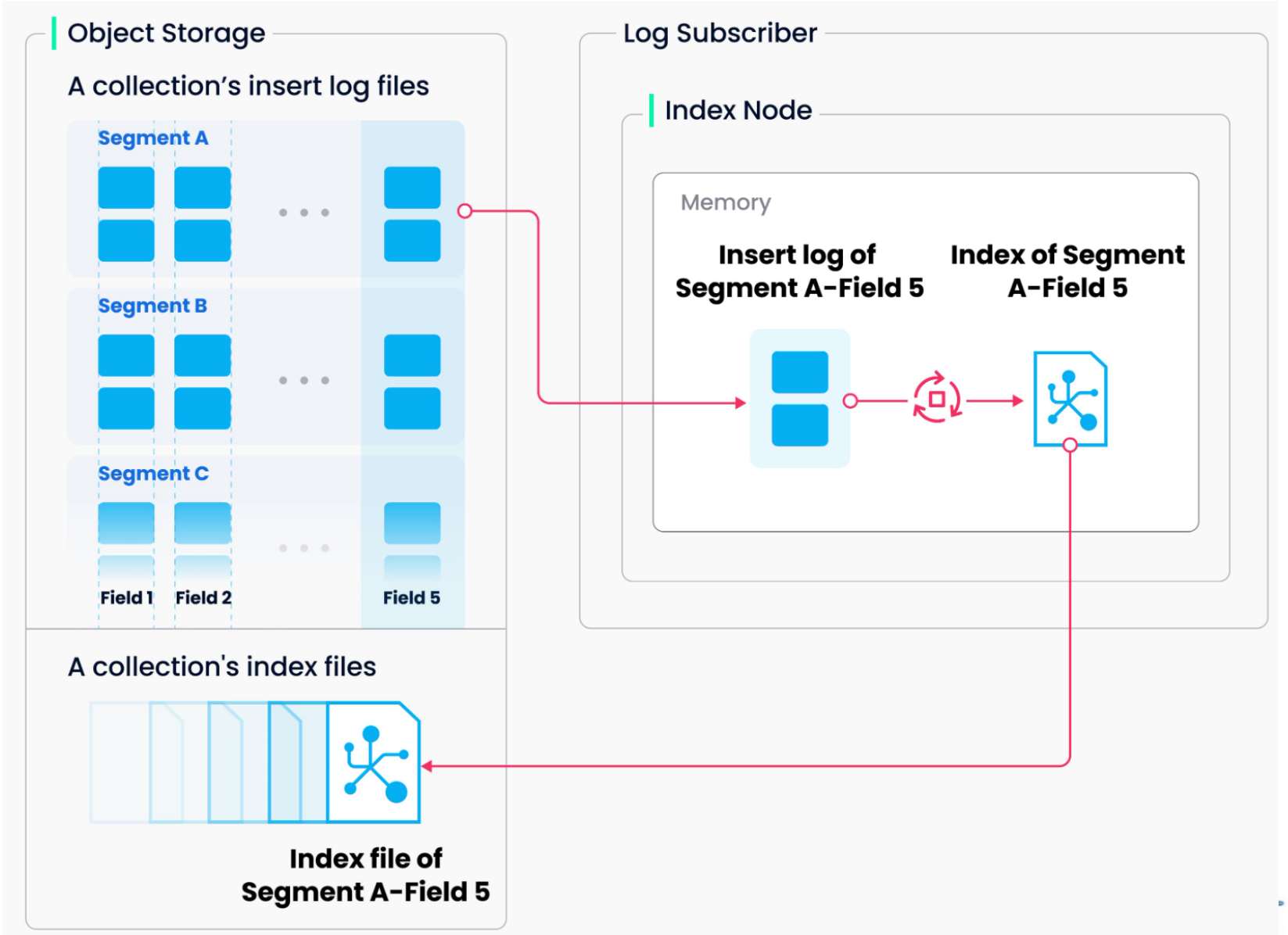
5.4 数据处理流程

- 数据插入：
  - 每个集合可以分为若干分片，每个分片对应一个虚拟通道（vChannel）。插入或删除请求根据主键的哈希值路由到不同的分片，代理会为每个请求分配时间戳，确保数据请求的顺序正确。DML（Data Manipulation Language） 请求会写入日志序列，并最终由数据节点处理并存储。



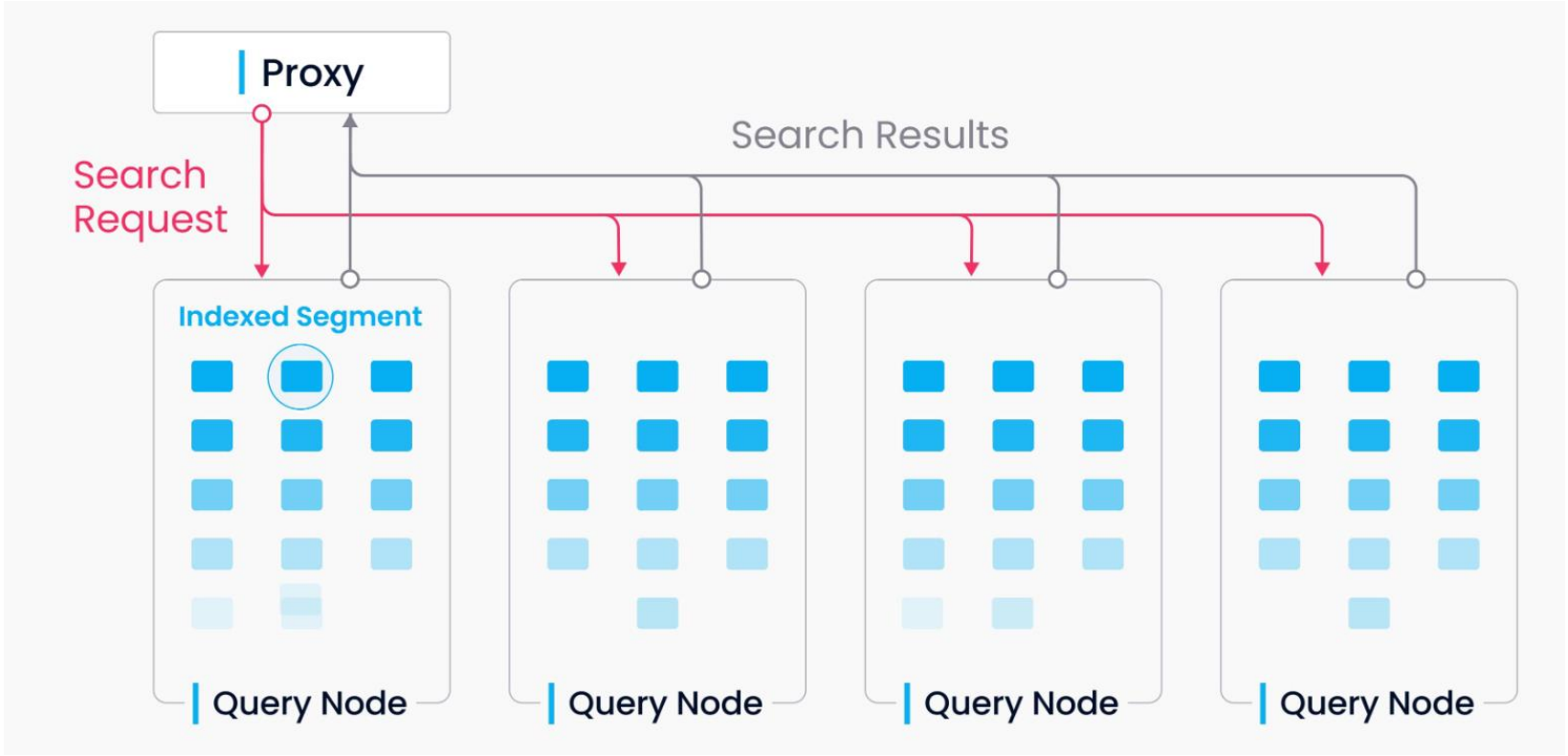
5.4 数据处理流程

- 索引构建：
  - Milvus 通过索引节点构建索引，每个集合分为多个分段，每个分段都有自己的索引。索引构建过程中需要加载日志快照，将其反序列化为向量数据，并进行训练后建立索引。索引构建是一个计算和内存密集型操作，常用 K-means 或图遍历等技术加速处理。



5.4 数据处理流程

- 数据查询：
  - 查询节点从对象存储中加载索引，并对指定的集合进行向量搜索。搜索请求会并发分发给所有查询节点，每个节点独立处理搜索请求，最后由代理汇总并返回最终结果。



5.5 应用案例

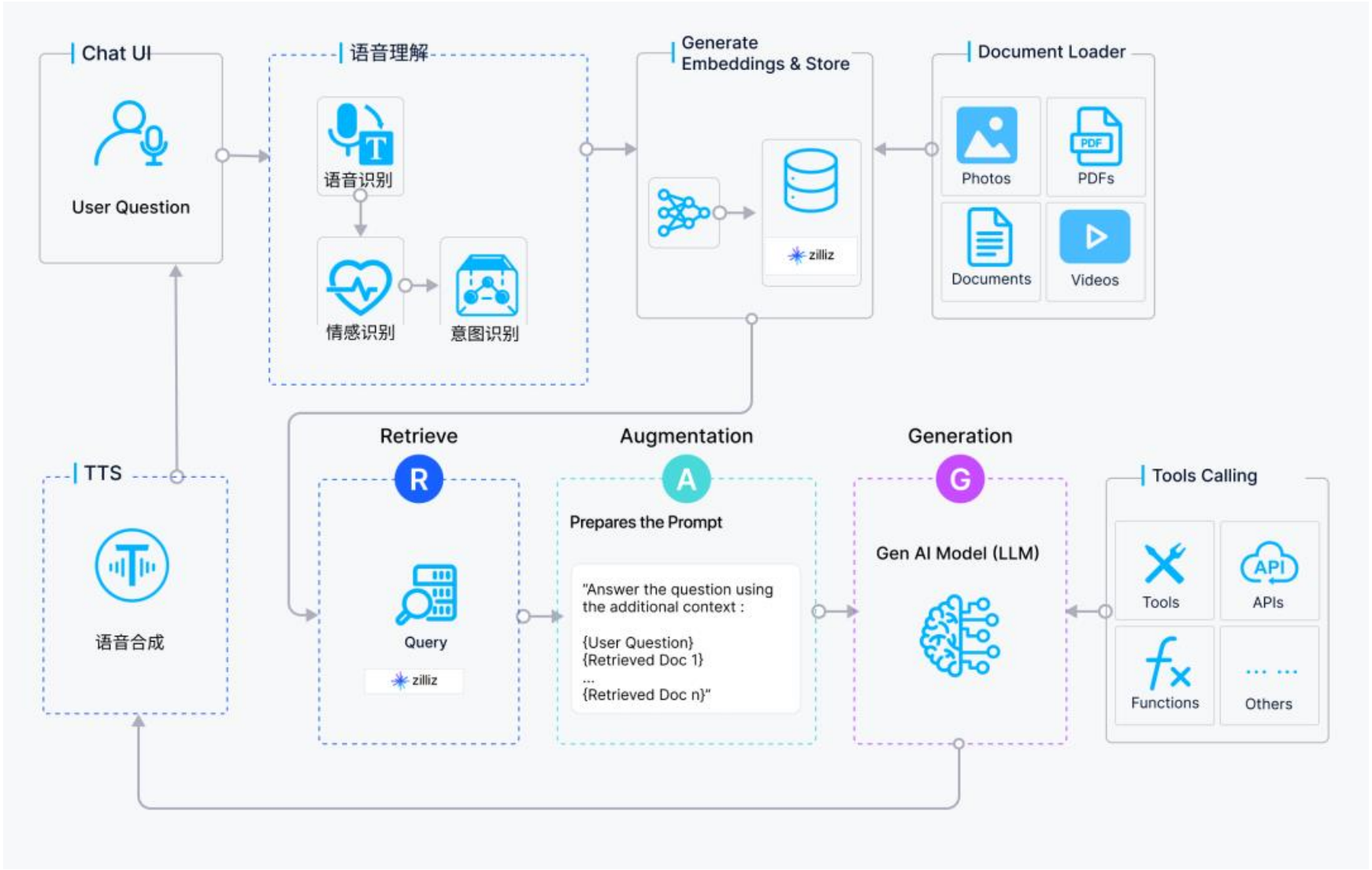
➢ 心辰Lingo：如何让AI更像人？

- 拥用情商
  - 向众多心理学专家、患者请教访谈，积累了大量优质的语料 -》情感计算和共情模块
  - 语音输出的延迟更低，同时表达更有情绪与语气起伏 -》端到端语音通话框架
  - 能够听得懂用户在说什么，也记得用户过去到底说过什么 -》RAG



5.5 应用案例

- 心辰Lingo：如何让AI更像人？



目录

- 1 数据存储技术回顾
- 2 经典数据仓库
- 3 数据湖
- 4 湖仓一体架构
- 5 向量数据库
- 6 小结

6 小结

- 经典数据仓库已发展30余载，虽光辉不再，但在数据分片、并行计算等方面留下了宝贵遗产；
- 数据湖作为下一代OLAP的基础设施，在野蛮生长多年后终形成标准范式；
- Lakehouse 结合了数据仓库和数据湖的优点，通过元数据层实现数据管理功能，通过查询引擎提升性能，通过开放文件格式直接访问，和云存储一样低成本。同时能够简化数据架构，提高数据的可靠性和新鲜度。
- AI时代到来，向量数据库破土而生，新一代云原生技术栈为大规模向量检索提供了全新解决方案。